```
import React, { useState, createContext, useContext, useEffect, useMemo, useDeferredValue, useCallback } from 'react';
import * as z from 'zod';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { Home, Users, Briefcase, Calendar, Link, Activity, AlertTriangle, Settings, Key, Zap, Code, Send, Shield, Clock, MessageSquare, LogOut } from 'lucide-react';

// --- I. UTILITY COMPONENTS (SHADCN/UI SIMULATION) ---

const Button = React.forwardRef(({ children, className = '', variant = 'primary', size = 'default', disabled, ...props }, ref) => {
    const baseStyle = 'inline-flex items-center justify-center rounded-lg font-medium transition-colors focus:outline-none focus:ring-2 focus:ring-offset-2 disabled:opacity-50 disabled:cursor-not-allowed shadow-sm';
    const sizeStyle = size === 'default' ? 'h-10 px-4 py-2 text-sm' : size === 'sm' ? 'h-8 px-3 text-xs' : 'h-12 px-6 text-base';
    const variantStyle = variant === 'primary' ? 'bg-indigo-600 text-white hover:bg-indigo-700 focus:ring-indigo-500' : variant === 'secondary' ? 'bg-gray-100 text-gray-800 hover:bg-gray-200 focus:ring-gray-300' : variant === 'destructive' ? 'bg-red-600 text-white hover:bg-red-700 focus:ring-red-500' : 'bg-transparent text-indigo-600 hover:bg-gray-100 focus:ring-indigo-500';
    return <button ref={ref} className={`${baseStyle} ${sizeStyle} ${variantStyle} ${className}`} disabled={disabled} {...props}>{children}</button>;
});

const Input = React.forwardRef(({ id, label, type = 'text', className = '', ...props }, ref) => (
    <div className="space-y-1">
        {label && <label htmlFor={id} className="text-sm font-medium text-gray-700 block">{label}</label>}
        <input id={id} type={type} ref={ref} className={`w-full px-3 py-2 border border-gray-300 rounded-lg shadow-sm focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 ${className}`} {...props} />
    </div>
));

const Textarea = React.forwardRef(({ id, label, className = '', ...props }, ref) => (
    <div className="space-y-1">
        {label && <label htmlFor={id} className="text-sm font-medium text-gray-700 block">{label}</label>}
        <textarea id={id} ref={ref} rows="4" className={`w-full px-3 py-2 border border-gray-300 rounded-lg shadow-sm focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 ${className}`} {...props} />
    </div>
));
```

```
const Select = React.forwardRef(({ id, label, className = '', options = [], ...props }, ref) => (
  <div className="space-y-1">
    {label && <label htmlFor={id} className="text-sm font-medium text-gray-700
block">{label}</label>}
    <select id={id} ref={ref} className={`w-full px-3 py-2 border border-gray-300 bg-white
rounded-lg shadow-sm focus:outline-none focus:ring-indigo-500 focus:border-indigo-500
${className}`} {...props}>
      {options.map(option => (
        <option key={option.value} value={option.value} disabled={option.disabled}>
          {option.label}
        </option>
      ))}
    </select>
  </div>
));

const Checkbox = React.forwardRef(({ id, label, ...props }, ref) => (
  <div className="flex items-center space-x-2">
    <input id={id} type="checkbox" ref={ref} className="h-4 w-4 text-indigo-600 border-gray-
300 rounded focus:ring-indigo-500" {...props} />
    {label && <label htmlFor={id} className="text-sm text-gray-700">{label}</label>}
  </div>
));

const Switch = React.forwardRef(({ id, checked, onCheckedChange, label, ...props }, ref) => (
  <label htmlFor={id} className="flex items-center cursor-pointer space-x-2">
    <div className="relative">
      <input type="checkbox" id={id} ref={ref} checked={checked} onChange={e =>
onCheckedChange(e.target.checked)} className="sr-only" {...props} />
      <div className={`block w-10 h-6 rounded-full transition ${checked ? 'bg-indigo-600' :
'bg-gray-300'}`}></div>
      <div className={`dot absolute left-1 top-1 bg-white w-4 h-4 rounded-full transition
transform ${checked ? 'translate-x-4' : ''}`}></div>
    </div>
    {label && <span className="text-sm font-medium text-gray-700">{label}</span>}
  </label>
));


const Card = ({ children, title, description, className = '' }) => (
  <div className={`bg-white rounded-xl shadow-lg border border-gray-200 ${className}`}>
    {(title || description) && (
      <div className="p-5 border-b">
```

```
            {title && <h2 className="text-lg font-semibold text-gray-900">{title}</h2>}
            {description && <p className="text-sm text-gray-500">{description}</p>}
          </div>
        )}
        <div className="p-5">
          {children}
        </div>
    </div>
);

const Badge = ({ children, variant = 'default', className = '' }) => {
    const base = 'inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium';
    const styles = variant === 'success' ? 'bg-green-100 text-green-800' : variant === 'warning' ?
'bg-yellow-100 text-yellow-800' : variant === 'critical' ? 'bg-red-100 text-red-800' : variant ===
'info' ? 'bg-blue-100 text-blue-800' : variant === 'secondary' ? 'bg-gray-100 text-gray-800' : 'bg-
indigo-100 text-indigo-800';
    return <span className={`${base} ${styles} ${className}`}>{children}</span>;
};

const Separator = ({ className = '' }) => <div className={`h-px w-full bg-gray-200
${className}`}></div>;

const Toast = {
    _notify: (message, type) => console.log(`[TOAST - ${type.toUpperCase()}] ${message}`),
    success: (message) => Toast._notify(message, 'success'),
    error: (message) => Toast._notify(message, 'error'),
    info: (message) => Toast._notify(message, 'info'),
};

const DataTable = ({ columns, data }) => (
    <div className="overflow-x-auto rounded-lg border border-gray-200">
        <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
                <tr>
                    {columns.map((col, index) => (
                        <th key={index} className="px-6 py-3 text-left text-xs font-medium text-gray-
500 uppercase tracking-wider">
                            {col.header}
                        </th>
                    ))}
                </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
                {data.map((row, rowIndex) => (
```

```
                <tr key={rowIndex}>
                  {columns.map((col, colIndex) => (
                    <td key={colIndex} className="px-6 py-4 whitespace-nowrap text-sm text-
gray-900">
                      {col.cell(row)}
                    </td>
                  ))}
                </tr>
              ))}
            </tbody>
          </table>
          {data.length === 0 && (
            <div className="p-6 text-center text-gray-500">No records found.</div>
          )}
        </div>
);

const Dialog = ({ children, title, open, onClose, className = '' }) => {
  if (!open) return null;
  return (
    <div className="fixed inset-0 z-50 overflow-y-auto bg-gray-900 bg-opacity-50 flex items-
center justify-center p-4">
      <div className={`bg-white rounded-xl shadow-2xl w-full max-w-lg transform transition-
all ${className}`}>
        <div className="p-6 border-b flex justify-between items-center">
          <h3 className="text-xl font-semibold text-gray-900">{title}</h3>
          <button onClick={onClose} className="text-gray-400 hover:text-gray-600
transition">
            <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24
24" xmlns="http://www.w3.org/2000/svg"><path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M6 18L18 6M6 6l12 12"></path></svg>
          </button>
        </div>
        <div className="p-6">
          {children}
        </div>
      </div>
    </div>
  );
};

const DropdownMenu = ({ children, trigger, className = '' }) => {
  const [open, setOpen] = useState(false);
  const toggleOpen = () => setOpen(prev => !prev);
```

```
    const close = () => setOpen(false);

    return (
        <div className={`relative ${className}`}>
            <div onClick={toggleOpen} className="cursor-pointer">
                {trigger}
            </div>
            {open && (
                <div onBlur={close} className="absolute right-0 mt-2 w-48 rounded-md shadow-lg
bg-white ring-1 ring-black ring-opacity-5 focus:outline-none z-10" role="menu">
                    <div className="py-1" onClick={close}>
                        {children}
                    </div>
                </div>
            )}
        </div>
    );
};

const DropdownMenuItem = ({ children, onClick, className = '' }) => (
    <button onClick={onClick} className={`block w-full text-left px-4 py-2 text-sm text-gray-700
hover:bg-gray-100 ${className}`} role="menuitem">
        {children}
    </button>
);

const Avatar = ({ initials, size = 'md' }) => {
    const sizeClasses = size === 'md' ? 'h-10 w-10 text-lg' : 'h-8 w-8 text-sm';
    return (
        <div className={`inline-flex items-center justify-center rounded-full bg-indigo-500 text-
white font-semibold ${sizeClasses}`}>
            {initials}
        </div>
    );
};

// --- II. MOCK DATA, SCHEMAS, AND CONSTANTS ---

// MOCK DATA: ALL_PERMISSIONS from Slice 14.0
const ALL_PERMISSIONS = [
    'user:view_all', 'user:manage_users', 'user:manage_roles', 'user:manage_active',
    'settings:view', 'settings:manage',
    'job:create_edit', 'job:test_sync', 'job:manage_credentials',
    'link:create_edit', 'link:test_sync',
```

```
    'exec:view_all', 'exec:view_own',
    'alert:view_all', 'alert:acknowledge_all',
    'credential:manage',
];

// MOCK DATA: USERS (Slice 0.1, 1.0, 11.0, 14.0)
const MOCK_USERS = [
    { id: 'usr-admin-001', email: 'admin@harpy.com', firstName: 'Admin', lastName: 'User', role:
'admin', is_active: true, password: 'admin123', permissions: ALL_PERMISSIONS,
phoneNumber: '555-123-4567', notificationPreference: 'Email', quietModeEnabled: true,
quietHoursStart: '22:00', quietHoursEnd: '06:00', alertFilters: { severity: 'critical' } },
    { id: 'usr-manager-002', email: 'manager@harpy.com', firstName: 'Manager', lastName:
'Smith', role: 'manager', is_active: true, password: 'password123', permissions:
[...ALL_PERMISSIONS.filter(p => !p.includes('manage_roles') &&
!p.includes('manage_active'))], phoneNumber: '555-987-6543', notificationPreference: 'SMS',
quietModeEnabled: false, quietHoursStart: '00:00', quietHoursEnd: '00:00', alertFilters: {
severity: 'all' } },
    { id: 'usr-user-003', email: 'user@harpy.com', firstName: 'Regular', lastName: 'Joe', role:
'user', is_active: true, password: 'password123', permissions: ['exec:view_own', 'alert:view_all',
'job:test_sync'], phoneNumber: '', notificationPreference: 'None', quietModeEnabled: false,
quietHoursStart: '00:00', quietHoursEnd: '00:00', alertFilters: { severity: 'all' } },
    { id: 'usr-pending-004', email: 'pending@harpy.com', firstName: 'Pending', lastName: 'Invite',
role: 'user', is_active: false, password: 'temp_password', permissions: [], phoneNumber: '',
notificationPreference: 'None', quietModeEnabled: false, quietHoursStart: '00:00',
quietHoursEnd: '00:00', alertFilters: { severity: 'all' } },
];

// MOCK DATA: JOB TYPES (Slice 2.0, 16.0)
const JOB_TYPES = [
    { id: 1, label: 'Executable (Shell Command)' },
    { id: 2, label: 'API Call (HTTP Request)' },
    { id: 4, label: 'Python Script (Embedded)' },
    { id: 5, label: 'AI Agent (Orchestration)' },
];

// MOCK DATA: CREDENTIALS (Slice 17.0)
let MOCK_CREDENTIALS = [
    { credentialId: 101, name: 'Gemini API Key', type: 'API Key', created_by: 'admin@harpy.com'
},
    { credentialId: 102, name: 'External DB Auth', type: 'Database Secret', created_by:
'manager@harpy.com' },
];
let nextCredentialId = 103;
```

```
// MOCK DATA: JOBS (Slice 2.0, 16.0, 17.0)
let MOCK_JOBS = [
   { jobId: 1, name: 'Daily Cleanup Script', jobTypeId: 1, description: 'Clears /tmp directory',
executableConfig: { executable: '/bin/bash', parameters: '-c "rm -rf /tmp/*"', workingDirectory: '/' },
retryAttempts: 0, priority: 2, createdby: 'user@harpy.com', isactive: true, credentialId: null },
   { jobId: 2, name: 'Send Weekly Report', jobTypeId: 2, description: 'Calls external reporting
API', credentialId: 101, isactive: true, priority: 1, createdby: 'admin@harpy.com', apiConfig: { url:
'https://api.harpy.com/report', httpMethod: 'POST', authType: 'Bearer Token' } },
   { jobId: 3, name: 'Monitor Server Load', jobTypeId: 4, description: 'Python script to check
CPU/Mem', scriptConfig: { scriptContent: 'import psutil; print(psutil.cpu_percent())',
runtimeEnvironment: 'python3-env' }, credentialId: null, isactive: true, priority: 3, createdby:
'manager@harpy.com' },
];
let nextJobId = 4;

// MOCK DATA: SCHEDULES (Slice 3.0, 16.1)
let MOCK_SCHEDULES = [
   { scheduleId: 10, name: 'Daily 2 AM Run', pattern: 1, starttime: '02:00', timezone: 'UTC',
createdby: 'user@harpy.com', isactive: true, pattern_string: 'Every day at 02:00 UTC' },
   { scheduleId: 11, name: 'Mon-Fri 09:00', pattern: 2, starttime: '09:00', timezone:
'America/New_York', weekdays: '1,2,3,4,5', createdby: 'admin@harpy.com', isactive: true,
pattern_string: 'Every Mon, Tue, Wed, Thu, Fri at 09:00 EST' },
   { scheduleId: 12, name: 'Custom Cron 1st Day', pattern: 4, pattern_string: '0 0 1 * *',
createdby: 'admin@harpy.com', isactive: true, pattern_string_calculated: 'Cron: 0 0 1 * *' },
];
let nextScheduleId = 13;

// MOCK DATA: SCHEDULED JOBS (Slice 4.0, 15.0)
let MOCK_SCHEDULED_JOBS = [
   { scheduledJobId: 100, jobId: 1, scheduleId: 10, name: 'Cleanup Daily', description: 'User
cleanup job', priority: 2, createdby: 'user@harpy.com', isactive: true, alertOnFailure: true,
escalationEmail: 'ops@harpy.com' },
   { scheduledJobId: 101, jobId: 2, scheduleId: 11, name: 'Admin Weekly Report', description:
'Admin report link', priority: 1, createdby: 'admin@harpy.com', isactive: true, alertOnFailure: true,
escalationEmail: '' },
   { scheduledJobId: 102, jobId: 3, scheduleId: 12, name: 'Manager Monthly Monitor',
description: 'Manager monitor link', priority: 3, createdby: 'manager@harpy.com', isactive: true,
alertOnFailure: false, alertOnSuccess: true, escalationEmail: '' },
];
let nextScheduledJobId = 103;

// MOCK DATA: EXECUTIONS (Slice 6.0, 7.0)
let MOCK_EXECUTIONS = [
```

```
  { executionId: 1000, scheduledJobId: 100, jobId: 1, startTime: new Date(Date.now() - 3600000), endTime: new Date(Date.now() - 3590000), executionStatus: 'completed', duration: 10.0, exitCode: 0, output: 'Files cleaned successfully.', errorOutput: null, createdby: 'user@harpy.com' },
  { executionId: 1001, scheduledJobId: 101, jobId: 2, startTime: new Date(Date.now() - 7200000), endTime: new Date(Date.now() - 7190000), executionStatus: 'failed', duration: 10.0, exitCode: 1, output: 'API call failed.', errorOutput: '{"status": 500, "message": "External service timeout"}', createdby: 'admin@harpy.com' },
  { executionId: 1002, scheduledJobId: 102, jobId: 3, startTime: new Date(Date.now() - 10800000), endTime: new Date(Date.now() - 10700000), executionStatus: 'completed', duration: 100.0, exitCode: 0, output: 'CPU Load: 45%', errorOutput: null, createdby: 'manager@harpy.com' },
  { executionId: 1003, scheduledJobId: 101, jobId: 2, startTime: new Date(Date.now() - 100000), endTime: new Date(Date.now() - 50000), executionStatus: 'failed', duration: 50.0, exitCode: 1, output: 'Rate limit exceeded.', errorOutput: '429 TOO MANY REQUESTS', createdby: 'admin@harpy.com' }, // Failed job for dashboard KPI
];
let nextExecutionId = 1004;

// MOCK DATA: ALERTS (Slice 7.0)
let MOCK_ALERTS = [
  { alertId: 1, alertTypeId: 1, scheduledJobId: 101, executionId: 1001, severity: 'critical', title: 'Job Failed: Admin Weekly Report', message: 'API call failed.', createddate: new Date(Date.now() - 7000000), acknowledged: false, createdby: 'admin@harpy.com' },
  { alertId: 2, alertTypeId: 3, scheduledJobId: 102, executionId: 1002, severity: 'info', title: 'Job Succeeded: Manager Monthly Monitor', message: 'First successful run.', createddate: new Date(Date.now() - 10700000), acknowledged: true, acknowledgedBy: 'manager@harpy.com', createdby: 'manager@harpy.com' },
  { alertId: 3, alertTypeId: 1, scheduledJobId: 101, executionId: 1003, severity: 'critical', title: 'Job Failed: Rate Limit', message: 'Rate limit exceeded.', createddate: new Date(Date.now() - 40000), acknowledged: false, createdby: 'admin@harpy.com' },
];
let nextAlertId = 4;

// MOCK DATA: UTILS
const getMockUser = (email) => MOCK_USERS.find(u => u.email === email);
const getJob = (jobId) => MOCK_JOBS.find(j => j.jobId === jobId);
const getSchedule = (scheduleId) => MOCK_SCHEDULES.find(s => s.scheduleId === scheduleId);
const getScheduledJob = (scheduledJobId) => MOCK_SCHEDULED_JOBS.find(sj => sj.scheduledJobId === scheduledJobId);
const getAlert = (alertId) => MOCK_ALERTS.find(a => a.alertId === alertId);

// CONSTANTS (Slice 0.1, 4.0, 14.0)
```

```
const ROLE_OPTIONS = [
    { value: 'admin', label: 'Admin' },
    { value: 'manager', label: 'Manager' },
    { value: 'user', label: 'User' },
];

const PRIORITY_OPTIONS = [
    { value: 1, label: 'High' },
    { value: 2, label: 'Medium' },
    { value: 3, label: 'Low' },
];

const SCHEDULE_PATTERN_OPTIONS = [
    { value: 1, label: 'Daily' },
    { value: 2, label: 'Weekly' },
    { value: 3, label: 'Monthly' },
    { value: 4, label: 'Custom (Cron Expression)' },
];

const WEEKDAY_OPTIONS = [
    { value: 1, label: 'Mon' },
    { value: 2, label: 'Tue' },
    { value: 3, label: 'Wed' },
    { value: 4, label: 'Thu' },
    { value: 5, label: 'Fri' },
    { value: 6, label: 'Sat' },
    { value: 7, label: 'Sun' },
];

const NOTIFICATION_OPTIONS = [
    { value: 'Email', label: 'Email' },
    { value: 'SMS', label: 'SMS' },
    { value: 'None', label: 'None' },
];

const ALERT_SEVERITY_OPTIONS = [
    { value: 'critical', label: 'Critical' },
    { value: 'warning', label: 'Warning' },
    { value: 'info', label: 'Info' },
];

const NAV_LINKS = [
    { path: '/dashboard', label: 'Dashboard', icon: Home, permissions: ['exec:view_own',
'exec:view_all'] },
```

```
    { path: '/dashboard/jobs', label: 'Jobs', icon: Briefcase, permissions: ['job:create_edit'] },
    { path: '/dashboard/schedules', label: 'Schedules', icon: Calendar, permissions:
['job:create_edit'] },
    { path: '/dashboard/scheduled-jobs', label: 'Scheduled Jobs', icon: Link, permissions:
['link:create_edit'] },
    { path: '/dashboard/monitoring', label: 'Monitoring', icon: Activity, permissions:
['exec:view_own', 'exec:view_all'] },
    { path: '/dashboard/alerts', label: 'Alerts', icon: AlertTriangle, permissions: ['alert:view_all'] },
    { path: '/dashboard/users', label: 'Users', icon: Users, permissions: ['user:view_all'] },
    { path: '/dashboard/credentials', label: 'Credentials', icon: Key, permissions:
['credential:manage'] },
    { path: '/dashboard/settings', label: 'Settings', icon: Settings, permissions: ['settings:view'] },
];

// --- III. ZOD SCHEMAS (Refactored) ---

// Custom Cron Validation (Slice 16.1)
const validateCronStructure = (cronString) => {
    if (!cronString) return "Cron expression cannot be empty.";
    const parts = cronString.trim().split(/\s+/).filter(p => p.length > 0);
    if (parts.length !== 5) {
        return `Cron expression must contain exactly 5 space-separated fields. Found
${parts.length}.`;
    }
    return null; // Success
};

// --- Job Schemas (Slice 2.0, 16.0, 17.0) ---

const ApiConfigSchema = z.object({
    url: z.string().url('Must be a valid URL'),
    httpMethod: z.enum(['GET', 'POST', 'PUT', 'DELETE', 'PATCH']),
    authType: z.enum(['None', 'Bearer Token', 'Basic Auth']).default('None'),
    expectedStatusCodes: z.array(z.number().int().min(100).max(599)).optional(),
});

const ExecutableConfigSchema = z.object({
    executable: z.string().min(1, 'Executable path is required').max(500),
    parameters: z.string().max(2000).optional().or(z.literal('')),
    workingDirectory: z.string().max(500).optional().or(z.literal('')),
});

const ScriptConfigSchema = z.object({
    scriptContent: z.string().min(10, 'Script content is required'),
```

```
    runtimeEnvironment: z.string().optional().or(z.literal('')),
});

const AIConfigSchema = z.object({
    modelId: z.string().min(3, 'Model ID is required'),
    agentConfig: z.string().optional().or(z.literal('')),
    runtimeEnvironment: z.string().optional().or(z.literal('')),
});

const JobCreateSchema = z.object({
    jobTypeId: z.number().int(),
    name: z.string().min(3, 'Name is required').max(100),
    description: z.string().optional().or(z.literal('')),
    timeout: z.number().int().min(0, 'Timeout must be >= 0').default(60),
    retryAttempts: z.number().int().min(0).max(10).default(0),
    retryDelay: z.number().int().min(0).max(3600).default(5),
    priority: z.number().int().min(1).max(3).default(2),
    isactive: z.boolean().default(true),
    credentialId: z.number().int().nullable().optional(),
    authMechanism: z.enum(['Runner Service Account', 'Specific Credential']).default('Runner
Service Account'),

    // Conditional Schemas (Slice 16.0)
    executableConfig: ExecutableConfigSchema.optional(),
    apiConfig: ApiConfigSchema.optional(),
    scriptConfig: ScriptConfigSchema.optional(),
    aiConfig: AIConfigSchema.optional(),
}).refine(data => {
    // Validation for Executable (1)
    if (data.jobTypeId === 1) return !!data.executableConfig?.executable;
    // Validation for API Call (2)
    if (data.jobTypeId === 2) return !!data.apiConfig?.url;
    // Validation for Python Script (4)
    if (data.jobTypeId === 4) return !!data.scriptConfig?.scriptContent;
    // Validation for AI Agent (5)
    if (data.jobTypeId === 5) return !!data.aiConfig?.modelId;
    return true;
}, { message: "Configuration missing for selected Job Type.", path: ['jobTypeId'] });

// --- Schedule Schemas (Slice 3.0, 16.1) ---

const TimeSchema = z.string().regex(/^([0-1]\d|2[0-3]):([0-5]\d)$/, 'Must be HH:MM format');

const ScheduleCreateSchema = z.object({
```

```
    name: z.string().min(3).max(100),
    description: z.string().optional().or(z.literal('')),
    pattern: z.number().int().min(1).max(4),
    timezone: z.string().default('UTC'),
    effectiveDate: z.string().optional().or(z.literal('')),
    expiryDate: z.string().nullable().optional(),
    isactive: z.boolean().default(true),

    // Pattern-specific fields
    starttime: TimeSchema.optional(),
    weekdays: z.string().optional().or(z.literal('')),
    dayofmonth: z.number().int().min(1).max(31).nullable().optional(),
    weeknumber: z.number().int().min(1).max(5).nullable().optional(),
    pattern_string: z.string().optional().or(z.literal('')), // Used for Cron Expression

}).refine(data => {
    // Validation for Daily/Weekly (1, 2)
    if (data.pattern === 1 || data.pattern === 2) return !!data.starttime;
    // Validation for Monthly (3)
    if (data.pattern === 3) return !!data.starttime && (!!data.dayofmonth || !!data.weeknumber);
    // Validation for Custom Cron (4) (Slice 16.1)
    if (data.pattern === 4) return validateCronStructure(data.pattern_string) === null;

    return true;
}, { message: "Timing configuration is required for the selected pattern.", path: ['starttime'] });

// --- Scheduled Job Schemas (Slice 4.0, 15.0) ---

const ScheduledJobCreateSchema = z.object({
    jobId: z.union([z.number().int().min(1), z.string().min(1, 'Job is required')]),
    scheduleId: z.union([z.number().int().min(1), z.string().min(1, 'Schedule is required')]),
    name: z.string().min(3).max(100),
    description: z.string().optional().or(z.literal('')),
    priority: z.number().int().min(1).max(3).default(2),
    maxConcurrentRuns: z.number().int().min(1).default(1),
    queueTimeout: z.number().int().min(0).default(60),
    isactive: z.boolean().default(true),

    // Alerting (Slice 4.0, 15.0)
    alertOnFailure: z.boolean().default(true),
    alertOnSuccess: z.boolean().default(false),
    alertOnDelay: z.boolean().default(true),
    delayThreshold: z.number().int().min(0).default(15),
    escalationEmail: z.string().email('Invalid email format').optional().or(z.literal('')), // Slice 15.0
```

```
});

// --- User Profile Schemas (Slice 1.0, 11.0) ---

const ProfileUpdateSchema = z.object({
    firstName: z.string().min(1, 'First Name is required'),
    lastName: z.string().min(1, 'Last Name is required'),
    phoneNumber: z.string().optional().or(z.literal('')),
    notificationPreference: z.enum(['Email', 'SMS', 'None']),
    // Slice 15.0 Quiet Hours
    quietModeEnabled: z.boolean().default(false),
    quietHoursStart: TimeSchema.optional().or(z.literal('')),
    quietHoursEnd: TimeSchema.optional().or(z.literal('')),
    // Slice 15.0 Alert Filters
    alertFilters: z.object({
        severity: z.enum(['critical', 'warning', 'info', 'all']).default('all'),
    }).default({ severity: 'all' }),
}).refine(data => {
    if (data.quietModeEnabled) return !!data.quietHoursStart && !!data.quietHoursEnd;
    return true;
}, { message: "Start and End times are required when Quiet Mode is enabled.", path:
['quietHoursStart'] });


const UserInviteSchema = z.object({
    email: z.string().email('Invalid email format'),
    role: z.enum(['admin', 'manager', 'user']),
});

const CredentialSchema = z.object({
    name: z.string().min(3, 'Name is required'),
    type: z.string().min(3, 'Type is required'),
    encryptedValue: z.string().min(10, 'Encrypted Value is required'),
});

// --- IV. AUTH CONTEXT & MOCK API ---

const AuthContext = createContext();

const formatDuration = (seconds) => {
    if (typeof seconds !== 'number' || isNaN(seconds)) return 'N/A';
    const minutes = Math.floor(seconds / 60);
    const remainingSeconds = seconds % 60;
```

```
    if (minutes > 0) return `${minutes}m ${remainingSeconds.toFixed(1)}s`;
    return `${seconds.toFixed(2)}s`;
};

const formatTime = (date) => date ? new Date(date).toLocaleString() : 'N/A';

const generatePatternString = (schedule) => {
    if (schedule.pattern === 4) return `Cron: ${schedule.pattern_string}`;
    let base = `Every day at ${schedule.starttime} ${schedule.timezone}`;
    if (schedule.pattern === 2 && schedule.weekdays) {
        const days = schedule.weekdays.split(',').map(d => WEEKDAY_OPTIONS.find(w =>
w.value == d)?.label).filter(Boolean).join(', ');
        base = `Every ${days} at ${schedule.starttime} ${schedule.timezone}`;
    } else if (schedule.pattern === 3) {
        if (schedule.dayofmonth) base = `Day ${schedule.dayofmonth} of every month at
${schedule.starttime} ${schedule.timezone}`;
        else if (schedule.weeknumber && schedule.weekdays) base = `The
${schedule.weeknumber} week day of the month at ${schedule.starttime}
${schedule.timezone}`;
    }
    return base;
};

const dispatchAlertNotifications = (alert, jobOwnerEmail, scheduledJob) => {
    const jobOwner = MOCK_USERS.find(u => u.email === jobOwnerEmail);
    const method = jobOwner?.notificationPreference || 'None';
    const contact = method === 'Email' ? jobOwner.email : method === 'SMS' ?
jobOwner.phoneNumber : 'N/A';
    const severity = alert.severity;

    // Slice 15.0: Escalation Check
    if (severity === 'critical' && scheduledJob.escalationEmail) {
        Toast.error(`[ESCALATION DISPATCH] Critical alert sent to escalation group:
${scheduledJob.escalationEmail}`);
    }

    // Slice 15.0: Quiet Hours Check (Mock Logic)
    if (jobOwner?.quietModeEnabled && severity !== 'critical') {
        // In a real app, we check NOW against quietHoursStart/End
        console.warn(`[NOTIFICATION SUPPRESSION] User ${jobOwner.email} is in Quiet Mode.
Non-critical alert suppressed/deferred.`);
        return;
    }
```

```
    // Standard Dispatch
    if (method !== 'None') {
        Toast.info(`[NOTIFICATION DISPATCH] Alert sent via ${method} to ${contact} for job
${scheduledJob.name}.`);
    } else {
        console.log(`[NOTIFICATION INFO] Alert for ${scheduledJob.name} generated, but owner
opted out of notifications.`);
    }
};

const createMockExecution = (scheduledJob) => {
    const status = Math.random() < 0.9 ? 'completed' : 'failed';
    const duration = Math.random() * 100;
    const execution = {
        executionId: nextExecutionId++,
        scheduledJobId: scheduledJob.scheduledJobId,
        jobId: scheduledJob.jobId,
        startTime: new Date(),
        endTime: new Date(Date.now() + duration * 1000),
        executionStatus: status,
        duration: duration,
        exitCode: status === 'completed' ? 0 : 1,
        output: status === 'completed' ? `Job finished successfully in ${formatDuration(duration)}.`
: `Job failed: API timeout.`,
        errorOutput: status === 'completed' ? null : `Error details: 504 Gateway Timeout.`,
        createdby: scheduledJob.createdby,
    };
    MOCK_EXECUTIONS.push(execution);

    // Slice 7.0: Alert generation
    if (status === 'failed' && scheduledJob.alertOnFailure) {
        const alert = {
            alertId: nextAlertId++,
            alertTypeId: 1, // Failure
            scheduledJobId: scheduledJob.scheduledJobId,
            executionId: execution.executionId,
            severity: 'critical',
            title: `Job Failed: ${scheduledJob.name}`,
            message: execution.errorOutput,
            createddate: new Date(),
            acknowledged: false,
            createdby: scheduledJob.createdby,
        };
        MOCK_ALERTS.push(alert);
```

```
        dispatchAlertNotifications(alert, scheduledJob.createdby, scheduledJob);
    }
};

const AuthProvider = ({ children }) => {
    const [currentUser, setCurrentUser] = useState(null);
    const [page, setPage] = useState('/auth/login');

    const can = useMemo(() => (permissionKey) => {
        if (!currentUser) return false;
        return currentUser.permissions.includes(permissionKey);
    }, [currentUser]);

    // RLS Mock Check
    const isAuthorized = useCallback((item, ownerKey = 'createdby') => {
        if (!currentUser) return false;
        return item[ownerKey] === currentUser.email || currentUser.role === 'admin' ||
currentUser.role === 'manager';
    }, [currentUser]);

    // --- MOCK API IMPLEMENTATION ---

    const api = useMemo(() => ({
        // --- AUTH API (Slice 0.1, 12.0) ---
        signIn: async ({ email, password }) => {
            const user = getMockUser(email);
            if (user && user.password === password) {
                if (!user.is_active) throw new Error('Account is inactive.');
                setCurrentUser({ ...user });
                Toast.success('Login successful!');
                return user;
            }
            throw new Error('Invalid credentials.');
        },
        signOut: async () => {
            setCurrentUser(null);
            setPage('/auth/login');
            Toast.info('Logged out.');
        },
        requestPasswordReset: async ({ email }) => {
            // Always return a generic success message to prevent user enumeration attacks (A07)
            if (getMockUser(email)) {
                Toast.info("Simulated: Password reset link sent to console.");
            }
```

```
        return { success: true, message: "If an account exists for that email, a reset link has been sent." };
    },

    // --- USER API (Slice 1.0, 11.0, 14.0) ---
    getUsers: async (filter = '') => {
        const isPermitted = can('user:view_all');

        const users = MOCK_USERS.filter(u =>
            isPermitted || u.email === currentUser.email
        ).filter(u =>
            filter === '' ||
            u.email.toLowerCase().includes(filter.toLowerCase()) ||
            u.firstName.toLowerCase().includes(filter.toLowerCase()) ||
            u.lastName.toLowerCase().includes(filter.toLowerCase())
        );
        return users;
    },
    inviteUser: async (data) => {
        if (!can('user:manage_users')) throw new Error('Forbidden: Insufficient permissions.');
        if (getMockUser(data.email)) throw new Error('User already exists.');
        const newUser = { id: `usr-new-${Math.random().toString(36).substring(7)}`, password: 'temp_password', is_active: false, permissions: [], ...data };
        MOCK_USERS.push(newUser);
        Toast.success(`Invite sent to ${data.email}.`);
        return newUser;
    },
    updateUser: async (id, data) => {
        if (!can('user:manage_users')) throw new Error('Forbidden: Insufficient permissions.');
        const index = MOCK_USERS.findIndex(u => u.id === id);
        if (index === -1) throw new Error('User not found.');

        // Prevent unauthorized role/active changes
        if (!can('user:manage_roles')) delete data.role;
        if (!can('user:manage_active')) delete data.is_active;

        MOCK_USERS[index] = { ...MOCK_USERS[index], ...data };
        Toast.success(`User ${MOCK_USERS[index].email} updated.`);
        return MOCK_USERS[index];
    },
    updateProfile: async (data) => {
        const index = MOCK_USERS.findIndex(u => u.id === currentUser.id);
        if (index === -1) throw new Error('User not found.');
```

```
            // Server-side RLS: filter out restricted fields (role, id, email)
            const allowedUpdates = {
                firstName: data.firstName,
                lastName: data.lastName,
                phoneNumber: data.phoneNumber,
                notificationPreference: data.notificationPreference,
                quietModeEnabled: data.quietModeEnabled,
                quietHoursStart: data.quietHoursStart,
                quietHoursEnd: data.quietHoursEnd,
                alertFilters: data.alertFilters,
            };

            MOCK_USERS[index] = { ...MOCK_USERS[index], ...allowedUpdates };
            setCurrentUser(MOCK_USERS[index]);
            Toast.success('Profile updated successfully.');
            return MOCK_USERS[index];
        },

        // --- CREDENTIALS API (Slice 17.0) ---
        getCredentials: async () => {
            if (!can('credential:manage')) throw new Error('Forbidden: Must have credential:manage
permission.');
            return MOCK_CREDENTIALS;
        },
        getCredentialOptions: async () => {
            if (!can('credential:manage')) return [];
            return MOCK_CREDENTIALS.map(c => ({ value: c.credentialId, label: `${c.name}
(${c.type})` }));
        },
        createCredential: async (data) => {
            if (!can('credential:manage')) throw new Error('Forbidden: Must have credential:manage
permission.');
            const newCred = { credentialId: nextCredentialId++, created_by: currentUser.email,
...data };
            MOCK_CREDENTIALS.push(newCred);
            Toast.success('Credential created.');
            return newCred;
        },
        deleteCredential: async (id) => {
            if (!can('credential:manage')) throw new Error('Forbidden: Must have credential:manage
permission.');
            MOCK_CREDENTIALS = MOCK_CREDENTIALS.filter(c => c.credentialId !== id);
            Toast.success('Credential deleted.');
        },
```

```
    // --- JOB API (Slice 2.0, 16.0, 17.0) ---
    getJobs: async (filter = '') => {
        const jobsWithDetails = MOCK_JOBS.map(j => ({
            ...j,
            jobTypeLabel: JOB_TYPES.find(t => t.id === j.jobTypeId)?.label,
            credentialName: MOCK_CREDENTIALS.find(c => c.credentialId ===
j.credentialId)?.name,
        }));

        const filtered = jobsWithDetails.filter(j =>
            isAuthorized(j) && (
                filter === '' ||
                j.name.toLowerCase().includes(filter.toLowerCase()) ||
                j.description?.toLowerCase().includes(filter.toLowerCase())
            )
        );
        return filtered;
    },
    getJob: async (id) => {
        const job = MOCK_JOBS.find(j => j.jobId == id);
        if (!job || !isAuthorized(job)) throw new Error('Job not found or access denied.');
        return job;
    },
    createJob: async (data) => {
        if (!can('job:create_edit')) throw new Error('Forbidden: Must have job:create_edit
permission.');
        const newJob = { jobId: nextJobId++, createdby: currentUser.email, ...data };
        MOCK_JOBS.push(newJob);
        Toast.success('Job created successfully.');
        return newJob;
    },
    updateJob: async (id, data) => {
        const index = MOCK_JOBS.findIndex(j => j.jobId == id);
        if (index === -1) throw new Error('Job not found.');
        if (!isAuthorized(MOCK_JOBS[index], 'createdby')) throw new Error('Access denied to
update job.');
        MOCK_JOBS[index] = { ...MOCK_JOBS[index], ...data };
        Toast.success('Job updated successfully.');
        return MOCK_JOBS[index];
    },
    deleteJob: async (id) => {
        const job = MOCK_JOBS.find(j => j.jobId == id);
```

```
        if (!job || !isAuthorized(job, 'createdby')) throw new Error('Job not found or access
denied.');
        MOCK_JOBS = MOCK_JOBS.filter(j => j.jobId != id);
        Toast.success('Job deleted.');
    },
    testJob: async (jobId) => {
        if (!can('job:test_sync')) throw new Error('Forbidden: Must have job:test_sync
permission.');
        const job = MOCK_JOBS.find(j => j.jobId == jobId);
        if (!job || !isAuthorized(job)) throw new Error('Job not found or access denied.');
        Toast.info(`Simulating synchronous test run for ${job.name}...`);
        await new Promise(resolve => setTimeout(resolve, 1500)); // Simulate delay
        const duration = Math.random() * 50;
        const isSuccess = Math.random() < 0.7;
        const execution = {
            executionId: nextExecutionId++,
            jobId: job.jobId,
            startTime: new Date(),
            endTime: new Date(Date.now() + duration * 1000),
            executionStatus: isSuccess ? 'completed' : 'failed',
            duration: duration,
            exitCode: isSuccess ? 0 : 1,
            output: `Test completed. Duration: ${formatDuration(duration)}.`,
            errorOutput: isSuccess ? null : `Test failed due to mock network error.`,
            createdby: currentUser.email,
        };
        MOCK_EXECUTIONS.push(execution);
        return execution;
    },

    // --- SCHEDULE API (Slice 3.0, 16.1) ---
    getSchedules: async (filter = '') => {
        const schedulesWithDetails = MOCK_SCHEDULES.map(s => ({
            ...s,
            patternLabel: generatePatternString(s),
        }));

        const filtered = schedulesWithDetails.filter(s =>
            isAuthorized(s) && (
                filter === '' ||
                s.name.toLowerCase().includes(filter.toLowerCase()) ||
                s.patternLabel.toLowerCase().includes(filter.toLowerCase())
            )
        );
```

```
            return filtered;
        },
        getSchedule: async (id) => {
            const schedule = getSchedule(id);
            if (!schedule || !isAuthorized(schedule)) throw new Error('Schedule not found or access
denied.');
            return schedule;
        },
        createSchedule: async (data) => {
            if (!can('job:create_edit')) throw new Error('Forbidden: Must have job:create_edit
permission.');
            const pattern_string = data.pattern === 4 ? data.pattern_string :
generatePatternString(data);
            const newSchedule = { scheduleId: nextScheduleId++, createdby: currentUser.email,
...data, pattern_string };
            MOCK_SCHEDULES.push(newSchedule);
            Toast.success('Schedule created successfully.');
            return newSchedule;
        },
        updateSchedule: async (id, data) => {
            const index = MOCK_SCHEDULES.findIndex(s => s.scheduleId == id);
            if (index === -1) throw new Error('Schedule not found.');
            if (!isAuthorized(MOCK_SCHEDULES[index], 'createdby')) throw new Error('Access
denied to update schedule.');
            const pattern_string = data.pattern === 4 ? data.pattern_string :
generatePatternString(data);
            MOCK_SCHEDULES[index] = { ...MOCK_SCHEDULES[index], ...data, pattern_string };
            Toast.success('Schedule updated successfully.');
            return MOCK_SCHEDULES[index];
        },
        deleteSchedule: async (id) => {
            const schedule = getSchedule(id);
            if (!schedule || !isAuthorized(schedule, 'createdby')) throw new Error('Schedule not found
or access denied.');
            MOCK_SCHEDULES = MOCK_SCHEDULES.filter(s => s.scheduleId != id);
            Toast.success('Schedule deleted.');
        },

        // --- SCHEDULED JOB API (Slice 4.0, 15.0) ---
        getScheduledJobs: async () => {
            const scheduledJobsWithDetails = MOCK_SCHEDULED_JOBS.map(sj => {
                const job = getJob(sj.jobId);
                const schedule = getSchedule(sj.scheduleId);
                return {
```

```
                ...sj,
                jobName: job?.name,
                scheduleName: schedule?.name,
            };
        }).filter(sj => sj.jobName && sj.scheduleName && isAuthorized(sj));
        return scheduledJobsWithDetails;
    },
    getScheduledJob: async (id) => {
        const sj = getScheduledJob(id);
        if (!sj) throw new Error('Scheduled Job not found.');
        const job = getJob(sj.jobId);
        const schedule = getSchedule(sj.scheduleId);
        if (!job || !schedule || !isAuthorized(sj)) throw new Error('Access denied or job/schedule
missing.');
        return { ...sj, jobName: job.name, scheduleName: schedule.name };
    },
    createScheduledJob: async (data) => {
        if (!can('link:create_edit')) throw new Error('Forbidden: Must have link:create_edit
permission.');
        const job = getJob(data.jobId);
        const schedule = getSchedule(data.scheduleId);
        if (!job || !schedule) throw new Error('Job or Schedule ID not found.');

        // A01 IDOR Check (Slice 4.0)
        if (currentUser.role !== 'admin' && (!isAuthorized(job, 'createdby') ||
!isAuthorized(schedule, 'createdby'))) {
            throw new Error('Access denied: Cannot link job/schedule not owned by you.');
        }

        const newScheduledJob = { scheduledJobId: nextScheduledJobId++, createdby:
currentUser.email, ...data };
        MOCK_SCHEDULED_JOBS.push(newScheduledJob);
        Toast.success('Scheduled Job link created.');

        // Auto-create initial execution for demo
        createMockExecution(newScheduledJob);

        return newScheduledJob;
    },
    updateScheduledJob: async (id, data) => {
        const index = MOCK_SCHEDULED_JOBS.findIndex(sj => sj.scheduledJobId == id);
        if (index === -1) throw new Error('Scheduled Job not found.');
        if (!isAuthorized(MOCK_SCHEDULED_JOBS[index], 'createdby')) throw new
Error('Access denied to update link.');
```

```
        MOCK_SCHEDULED_JOBS[index] = { ...MOCK_SCHEDULED_JOBS[index], ...data };
        Toast.success('Scheduled Job updated successfully.');
        return MOCK_SCHEDULED_JOBS[index];
    },
    deleteScheduledJob: async (id) => {
        const sj = getScheduledJob(id);
        if (!sj || !isAuthorized(sj, 'createdby')) throw new Error('Scheduled Job not found or
access denied.');
        MOCK_SCHEDULED_JOBS = MOCK_SCHEDULED_JOBS.filter(sj =>
sj.scheduledJobId != id);
        Toast.success('Scheduled Job deleted.');
    },

    // --- EXECUTION API (Slice 6.0) ---
    getExecutions: async (filter = {}) => {
        const { jobId, status, query } = filter;
        const allExecutionsWithDetails = MOCK_EXECUTIONS.map(e => {
            const sj = getScheduledJob(e.scheduledJobId);
            const job = getJob(e.jobId);
            return { ...e, scheduledJobName: sj?.name, jobName: job?.name };
        }).filter(e => {
            // RLS Check (A01)
            const passesRls = isAuthorized(e);
            // Exec View All Override (Slice 14.0)
            const passesPermission = can('exec:view_all') || passesRls;
            if (!passesPermission) return false;

            // Filtering Logic
            if (jobId && e.jobId != jobId) return false;
            if (status && e.executionStatus !== status) return false;
            if (query && !e.jobName?.toLowerCase().includes(query.toLowerCase())) return false;
            return true;
        }).sort((a, b) => b.startTime.getTime() - a.startTime.getTime()); // Sort by time desc

        return allExecutionsWithDetails;
    },
    getExecution: async (id) => {
        const execution = MOCK_EXECUTIONS.find(e => e.executionId == id);
        if (!execution) throw new Error('Execution not found.');
        const sj = getScheduledJob(execution.scheduledJobId);
        const job = getJob(execution.jobId);

        // RLS Check (A01)
```

```
        const passesRls = isAuthorized(execution);
        // Exec View All Override (Slice 14.0)
        const passesPermission = can('exec:view_all') || passesRls;

        if (!passesPermission || !sj || !job) throw new Error('Execution not found or access
denied.');
        return { ...execution, scheduledJobName: sj.name, jobName: job.name };
    },

    // --- ALERT API (Slice 7.0) ---
    getAlerts: async (filter = {}) => {
        const { acknowledged, severity } = filter;
        const alertsWithDetails = MOCK_ALERTS.map(a => {
            const sj = getScheduledJob(a.scheduledJobId);
            return { ...a, scheduledJobName: sj?.name };
        }).filter(a => {
            // RLS Check (A01)
            const passesRls = isAuthorized(a);
            // Alert View All Override (Slice 14.0)
            const passesPermission = can('alert:view_all') || passesRls;
            if (!passesPermission) return false;

            // Filtering
            if (acknowledged !== undefined && a.acknowledged !== acknowledged) return false;
            if (severity && a.severity !== severity) return false;
            return true;
        }).sort((a, b) => b.createddate.getTime() - a.createddate.getTime());

        return alertsWithDetails;
    },
    acknowledgeAlert: async (id) => {
        const index = MOCK_ALERTS.findIndex(a => a.alertId == id);
        if (index === -1) throw new Error('Alert not found.');
        if (!isAuthorized(MOCK_ALERTS[index])) throw new Error('Access denied to
acknowledge alert.');

        // Check specific permission (Slice 14.0)
        if (MOCK_ALERTS[index].createdby !== currentUser.email &&
!can('alert:acknowledge_all')) {
            throw new Error('Forbidden: Cannot acknowledge alerts not created by you.');
        }

        MOCK_ALERTS[index] = {
            ...MOCK_ALERTS[index],
```

```
            acknowledged: true,
            acknowledgedBy: currentUser.email,
            acknowledgedDate: new Date(),
        };
        Toast.success('Alert acknowledged.');
        return MOCK_ALERTS[index];
    },

    // --- DASHBOARD API (Slice 9.0) ---
    getDashboardSummary: async () => {
        const now = Date.now();
        const twentyFourHoursAgo = now - 24 * 60 * 60 * 1000;

        const allScheduledJobs = MOCK_SCHEDULED_JOBS.filter(isAuthorized);
        const allExecutions = MOCK_EXECUTIONS.filter(e => can('exec:view_all') ||
isAuthorized(e));
        const allAlerts = MOCK_ALERTS.filter(a => can('alert:view_all') || isAuthorized(a));

        const activeSchedules = allScheduledJobs.filter(sj => sj.isactive).length;
        const failedJobs24h = allExecutions.filter(e => e.executionStatus === 'failed' &&
e.startTime.getTime() > twentyFourHoursAgo).length;
        const activeAlerts = allAlerts.filter(a => !a.acknowledged).length;

        const recentExecutions = allExecutions
            .sort((a, b) => b.startTime.getTime() - a.startTime.getTime())
            .slice(0, 5)
            .map(e => ({
                ...e,
                jobName: getJob(e.jobId)?.name,
                scheduledJobName: getScheduledJob(e.scheduledJobId)?.name
            }));

        const activeAlertList = allAlerts
            .filter(a => !a.acknowledged)
            .sort((a, b) => b.createddate.getTime() - a.createddate.getTime())
            .slice(0, 5)
            .map(a => ({ alertId: a.alertId, severity: a.severity, title: a.title }));

        const executionTrend = Array(7).fill(0).map((_, i) => {
            const day = new Date(now - (6 - i) * 24 * 60 * 60 * 1000);
            const dayStart = new Date(day.getFullYear(), day.getMonth(), day.getDate());
            const dayEnd = new Date(day.getFullYear(), day.getMonth(), day.getDate() + 1);
```

```
            const completed = allExecutions.filter(e => e.executionStatus === 'completed' &&
e.startTime >= dayStart && e.startTime < dayEnd).length;
            const failed = allExecutions.filter(e => e.executionStatus === 'failed' && e.startTime
>= dayStart && e.startTime < dayEnd).length;
            return { day: dayStart.toLocaleDateString('en-US', { weekday: 'short' }),
status_completed: completed, status_failed: failed };
        });

        return {
            activeSchedules,
            failedJobs24h,
            activeAlerts,
            recentExecutions,
            activeAlertList,
            executionTrend,
        };
    },

    // --- CONTEXT EXPORTS ---
    currentUser,
    can,
    setPage,
    page,

}), [currentUser, can, isAuthorized]);

// Initial Auth Middleware
useEffect(() => {
    const path = page.split('/')[2];
    const isAuthRoute = ['login', 'signup', 'reset-request'].includes(path);

    if (!currentUser && !isAuthRoute) {
        setPage('/auth/login');
    } else if (currentUser && isAuthRoute) {
        setPage('/dashboard');
    }
}, [currentUser, page, setPage]);


return (
    <AuthContext.Provider value={api}>
        {children}
    </AuthContext.Provider>
);
```

```
};

const useAuth = () => useContext(AuthContext);

// --- V. PAGE COMPONENTS ---

const Header = () => {
    const { currentUser, signOut, page, setPage, can } = useAuth();
    if (!currentUser) return null;

    const initials = `${currentUser.firstName[0]}${currentUser.lastName[0]}`;

    return (
        <header className="flex items-center justify-between p-4 bg-white border-b border-gray-
200 shadow-sm">
            <div className="flex items-center space-x-2">
                <Shield className="h-6 w-6 text-indigo-600" />
                <span className="text-xl font-semibold text-gray-800">Harpy Scheduler</span>
            </div>
            <div className="flex items-center space-x-4">
                <DropdownMenu trigger={<Avatar initials={initials} />} className="z-20">
                    <div className="px-4 py-3">
                        <p className="text-sm font-medium text-gray-900">{currentUser.firstName}
{currentUser.lastName}</p>
                        <p className="text-xs text-gray-500 truncate">{currentUser.email}</p>
                        <Badge variant="secondary" className="mt-
1">{currentUser.role.toUpperCase()}</Badge>
                    </div>
                    <Separator className="my-1" />
                    <DropdownMenuItem onClick={() => setPage('/dashboard/profile')}>My
Profile</DropdownMenuItem>
                    {can('settings:view') && (
                        <DropdownMenuItem onClick={() =>
setPage('/dashboard/settings')}>Settings</DropdownMenuItem>
                    )}
                    <DropdownMenuItem onClick={signOut} className="text-red-600">
                        <LogOut className="h-4 w-4 inline mr-2" />
                        Log Out
                    </DropdownMenuItem>
                </DropdownMenu>
            </div>
        </header>
    );
};
```

```
const Sidebar = () => {
   const { page, setPage, can } = useAuth();

   return (
      <aside className="w-56 flex flex-col bg-gray-800 text-white">
         <div className="p-4 flex items-center justify-center border-b border-gray-700">
            <Zap className="h-6 w-6 text-indigo-400" />
            <span className="ml-2 text-lg font-bold">Harpy</span>
         </div>
         <nav className="flex-1 space-y-2 p-4">
            {NAV_LINKS.filter(link => link.permissions.some(p => can(p))).map(link => (
               <button
                  key={link.path}
                  onClick={() => setPage(link.path)}
                  className={`flex items-center w-full px-3 py-2 text-sm font-medium rounded-lg
transition-colors ${page.startsWith(link.path) ? 'bg-indigo-600 text-white' : 'hover:bg-gray-700
text-gray-300'}`}
               >
                  <link.icon className="h-5 w-5 mr-3" />
                  {link.label}
               </button>
            ))}
         </nav>
      </aside>
   );
};

const AppLayout = ({ children }) => (
   <div className="flex h-screen bg-gray-50">
      <Sidebar />
      <div className="flex-1 flex flex-col overflow-hidden">
         <Header />
         <main className="flex-1 overflow-y-auto p-6">
            {children}
         </main>
      </div>
   </div>
);

const AuthLayout = ({ children }) => (
   <div className="min-h-screen flex items-center justify-center bg-gray-100 p-4">
      <div className="w-full max-w-md">
         <div className="text-center mb-8">
```

```
            <Shield className="h-10 w-10 text-indigo-600 mx-auto" />
            <h2 className="mt-4 text-3xl font-extrabold text-gray-900">Harpy Scheduler</h2>
          </div>
          <Card className="p-8">
            {children}
          </Card>
        </div>
      </div>
);

const UserManagementPage = () => {
    const { api, can, currentUser } = useAuth();
    const [users, setUsers] = useState([]);
    const [isLoading, setIsLoading] = useState(true);
    const [inviteDialogOpen, setInviteDialogOpen] = useState(false);
    const [filterQuery, setFilterQuery] = useState('');

    const deferredQuery = useDeferredValue(filterQuery);

    const fetchUsers = useCallback(async (query) => {
        setIsLoading(true);
        try {
            const data = await api.getUsers(query);
            setUsers(data);
        } catch (e) {
            Toast.error(`Failed to load users: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        // Only fetch if permitted, or if it's the initial load for the non-admin user (to see own profile)
        if (can('user:view_all') || (currentUser && !isLoading && users.length === 0)) {
            fetchUsers(deferredQuery);
        } else if (currentUser && isLoading) {
             fetchUsers(deferredQuery);
        }
    }, [deferredQuery, can, isLoading, fetchUsers, currentUser, users.length]);

    if (!can('user:view_all') && users.length === 0 && !isLoading) {
        return (
            <Card title="Access Denied" className="max-w-xl mx-auto">
```

```
            <p className="text-red-500">You do not have the permission (`user:view_all`) to
view the User Management console.</p>
          </Card>
        );
    }

    const columns = [
        { header: 'Name', cell: (user) => `${user.firstName} ${user.lastName}` },
        { header: 'Email', cell: (user) => user.email },
        { header: 'Role', cell: (user) => <Badge variant="info">{user.role}</Badge> },
        { header: 'Status', cell: (user) => <Badge variant={user.is_active ? 'success' :
'secondary'}>{user.is_active ? 'Active' : 'Pending'}</Badge> },
        {
            header: 'Actions',
            cell: (user) => (
                <DropdownMenu trigger={<Button size="sm"
variant="secondary">Actions</Button>}>
                    {can('user:manage_users') && <DropdownMenuItem onClick={() =>
Toast.info(`Editing ${user.email}`)}>Edit</DropdownMenuItem>}
                    {can('user:manage_active') && user.is_active && <DropdownMenuItem onClick={()
=> Toast.error(`Deactivating ${user.email}`)} className="text-red-
600">Deactivate</DropdownMenuItem>}
                    {can('user:manage_active') && !user.is_active && <DropdownMenuItem onClick={()
=> Toast.success(`Activating ${user.email}`)}>Activate</DropdownMenuItem>}
                </DropdownMenu>
            ),
        },
    ];

    return (
        <div className="space-y-6">
            <div className="flex justify-between items-center">
                <h1 className="text-3xl font-bold text-gray-900">User Management</h1>
                {can('user:manage_users') && (
                    <Button onClick={() => setInviteDialogOpen(true)}>+ Invite User</Button>
                )}
            </div>
            <Input
                placeholder="Search users by email or name..."
                value={filterQuery}
                onChange={(e) => setFilterQuery(e.target.value)}
            />
            <Card>
                {isLoading ? (
```

```
                <p className="text-center text-gray-500">Loading users...</p>
            ) : (
                <DataTable columns={columns} data={users} />
            )}
        </Card>
        <InviteUserDialog open={inviteDialogOpen} onClose={() => setInviteDialogOpen(false)}
onInvite={() => fetchUsers(filterQuery)} />
    </div>
  );
};

const InviteUserDialog = ({ open, onClose, onInvite }) => {
  const { api } = useAuth();
  const { register, handleSubmit, formState: { errors, isSubmitting }, reset } = useForm({
    resolver: zodResolver(UserInviteSchema),
    defaultValues: { role: 'user' }
  });

  const onSubmit = async (data) => {
    try {
      await api.inviteUser(data);
      Toast.success(`Invite sent to ${data.email}. User added as ${data.role}.`);
      onInvite();
      onClose();
      reset();
    } catch (e) {
      Toast.error(`Invite failed: ${e.message}`);
    }
  };

  return (
    <Dialog title="Invite New User" open={open} onClose={onClose}>
      <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
        <Input label="Email Address" id="email" {...register('email')} disabled={isSubmitting} />
        {errors.email && <p className="text-red-500 text-sm">{errors.email.message}</p>}

        <Select label="Role" id="role" options={ROLE_OPTIONS} {...register('role')}
disabled={isSubmitting} />
        {errors.role && <p className="text-red-500 text-sm">{errors.role.message}</p>}

        <div className="flex justify-end space-x-3 pt-2">
          <Button type="button" variant="secondary" onClick={onClose}
disabled={isSubmitting}>Cancel</Button>
          <Button type="submit" disabled={isSubmitting}>
```

```
                    {isSubmitting ? 'Sending...' : 'Send Invite'}
                </Button>
            </div>
        </form>
    </Dialog>
  );
};

const ProfilePage = () => {
  const { api, currentUser } = useAuth();
  const [isSubmitting, setIsSubmitting] = useState(false);
  const { register, handleSubmit, formState: { errors }, reset, watch } = useForm({
    resolver: zodResolver(ProfileUpdateSchema),
    defaultValues: {
      firstName: currentUser.firstName,
      lastName: currentUser.lastName,
      phoneNumber: currentUser.phoneNumber || '',
      notificationPreference: currentUser.notificationPreference || 'None',
      quietModeEnabled: currentUser.quietModeEnabled || false,
      quietHoursStart: currentUser.quietHoursStart || '00:00',
      quietHoursEnd: currentUser.quietHoursEnd || '00:00',
      alertFilters: currentUser.alertFilters || { severity: 'all' }
    }
  });

  const quietModeEnabled = watch('quietModeEnabled');

  const onSubmit = async (data) => {
    setIsSubmitting(true);
    try {
      await api.updateProfile(data);
      Toast.success('Profile updated successfully!');
      reset(data); // Reset form with new data
    } catch (e) {
      Toast.error(`Update failed: ${e.message}`);
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <Card title="My Profile & Notification Settings" className="max-w-3xl mx-auto">
      <form onSubmit={handleSubmit(onSubmit)} className="space-y-6">
        {/* Section 1: Account Details (Read-only) */}
```

```
            <div className="space-y-4">
              <h3 className="text-lg font-semibold border-b pb-2">Account Details</h3>
              <div className="grid grid-cols-2 gap-4">
                <Input label="Email" value={currentUser.email} readOnly disabled />
                <Input label="Role" value={currentUser.role} readOnly disabled />
                <Input label="User ID" value={currentUser.id} readOnly disabled
className="col-span-2" />
              </div>
            </div>

            <Separator />

            {/* Section 2: Personal Details */}
            <div className="space-y-4">
              <h3 className="text-lg font-semibold border-b pb-2">Personal Details</h3>
              <div className="grid grid-cols-2 gap-4">
                <Input label="First Name" id="firstName" {...register('firstName')} />
                {errors.firstName && <p className="text-red-500 text-sm col-span-
2">{errors.firstName.message}</p>}
                <Input label="Last Name" id="lastName" {...register('lastName')} />
                {errors.lastName && <p className="text-red-500 text-sm col-span-
2">{errors.lastName.message}</p>}
              </div>
            </div>

            <Separator />

            {/* Section 3: Notification Settings (Slice 11.0 & 15.0) */}
            <div className="space-y-4">
              <h3 className="text-lg font-semibold border-b pb-2">Notification Settings</h3>
              <div className="grid grid-cols-2 gap-4">
                <Select label="Delivery Method" id="notificationPreference"
options={NOTIFICATION_OPTIONS} {...register('notificationPreference')} />
                <Input label="Phone Number (for SMS)" id="phoneNumber"
{...register('phoneNumber')} placeholder="Optional: 555-123-4567" />
              </div>

              <h4 className="font-medium pt-2">Quiet Hours (Non-Critical Alert
Suppression)</h4>
              <div className="flex items-center space-x-4">
                <Switch id="quietModeEnabled" label="Enable Quiet Mode"
checked={quietModeEnabled} onCheckedChange={(val) => reset({ ...watch(),
quietModeEnabled: val })} />
              </div>
```

```
                {quietModeEnabled && (
                  <div className="grid grid-cols-2 gap-4 pt-2">
                    <Input label="Start Time" id="quietHoursStart" type="time"
{...register('quietHoursStart')} />
                    {errors.quietHoursStart && <p className="text-red-500 text-sm col-span-
2">{errors.quietHoursStart.message}</p>}
                    <Input label="End Time" id="quietHoursEnd" type="time"
{...register('quietHoursEnd')} />
                    {errors.quietHoursEnd && <p className="text-red-500 text-sm col-span-
2">{errors.quietHoursEnd.message}</p>}
                  </div>
                )}

                <h4 className="font-medium pt-4">Global Alert Filtering</h4>
                <p className="text-sm text-gray-500">Filter alerts by severity level. Alerts below
this level will be suppressed (overrides quiet mode).</p>
                <Select
                  label="Suppress Alerts Below Severity"
                  id="alertFilters.severity"
                  options={[{ value: 'all', label: 'Receive All Alerts' },
...ALERT_SEVERITY_OPTIONS]}
                  {...register('alertFilters.severity')}
                />
                {errors.alertFilters?.severity && <p className="text-red-500 text-sm col-span-
2">{errors.alertFilters.severity.message}</p>}


              </div>

              <div className="flex justify-end pt-4">
                <Button type="submit" disabled={isSubmitting}>
                  {isSubmitting ? 'Saving...' : 'Save Changes'}
                </Button>
              </div>
            </form>
          </Card>
    );
};

const RequestResetForm = ({ onResetSuccess }) => {
    const { api } = useAuth();
    const [isSubmitting, setIsSubmitting] = useState(false);
    const [success, setSuccess] = useState(false);
    const { register, handleSubmit, formState: { errors } } = useForm({
```

```
        resolver: zodResolver(z.object({ email: z.string().email('Invalid email format') }))
    });

    const onSubmit = async (data) => {
        setIsSubmitting(true);
        try {
            const result = await api.requestPasswordReset(data);
            Toast.info(result.message);
            setSuccess(true);
            onResetSuccess(result.message);
        } catch (e) {
            Toast.error(`Request failed: ${e.message}`);
        } finally {
            setIsSubmitting(false);
        }
    };

    if (success) {
        return (
            <div className="text-center p-4">
                <h3 className="text-lg font-medium text-green-600">Request Sent!</h3>
                <p className="text-gray-500 mt-2">Check your inbox for instructions to reset your
password.</p>
            </div>
        );
    }

    return (
        <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
            <h3 className="text-xl font-medium text-gray-900 mb-4">Reset Your Password</h3>
            <Input label="Email Address" id="reset-email" {...register('email')} placeholder="Enter
your email" disabled={isSubmitting} />
            {errors.email && <p className="text-red-500 text-sm">{errors.email.message}</p>}

            <div className="flex justify-end pt-2">
                <Button type="submit" disabled={isSubmitting}>
                    {isSubmitting ? 'Sending...' : 'Send Reset Link'}
                </Button>
            </div>
        </form>
    );
};

const LoginForm = ({ setAuthPage }) => {
```

```
    const { api } = useAuth();
    const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
        resolver: zodResolver(z.object({
            email: z.string().email('Invalid email format'),
            password: z.string().min(1, 'Password is required')
        }))
    });

    const onSubmit = async (data) => {
        try {
            await api.signIn(data);
        } catch (e) {
            Toast.error(e.message);
        }
    };

    return (
        <div className="space-y-6">
            <h3 className="text-xl font-medium text-gray-900 text-center">Sign in to your
account</h3>
            <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
                <Input label="Email Address" id="login-email" {...register('email')}
disabled={isSubmitting} />
                {errors.email && <p className="text-red-500 text-sm">{errors.email.message}</p>}

                <Input label="Password" id="login-password" type="password"
{...register('password')} disabled={isSubmitting} />
                {errors.password && <p className="text-red-500 text-
sm">{errors.password.message}</p>}

                <div className="flex justify-end">
                    <button type="button" onClick={() => setAuthPage('reset-request')}
className="text-sm text-indigo-600 hover:text-indigo-500">
                        Forgot Password?
                    </button>
                </div>

                <Button type="submit" className="w-full" disabled={isSubmitting}>
                    {isSubmitting ? 'Signing In...' : 'Sign In'}
                </Button>
            </form>
            <div className="text-sm text-center">
                Don't have an account?{' '}
```

```
            <button onClick={() => setAuthPage('signup')} className="font-medium text-indigo-
600 hover:text-indigo-500">
                Sign Up
            </button>
        </div>
        <div className="p-2 border rounded-lg text-xs text-gray-600">
            **Test Credentials:**<br />
            Admin: <code>admin@harpy.com</code> / <code>admin123</code><br />
            User: <code>user@harpy.com</code> / <code>password123</code>
        </div>
    </div>
  );
};

const SignUpForm = ({ setAuthPage }) => {
  const { api } = useAuth();
  const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
    resolver: zodResolver(z.object({
      email: z.string().email('Invalid email format'),
      firstName: z.string().min(1, 'First Name is required'),
      lastName: z.string().min(1, 'Last Name is required'),
      password: z.string().min(6, 'Password must be at least 6 characters'),
    }))
  });

  const onSubmit = async (data) => {
    try {
      // Mock sign up logic: creates user and signs them in
      const newUser = {
        id: `usr-new-${Math.random().toString(36).substring(7)}`,
        is_active: true,
        role: 'user',
        permissions: ['exec:view_own', 'alert:view_all'], // default permissions
        ...data,
      };
      MOCK_USERS.push(newUser);
      await api.signIn({ email: data.email, password: data.password });
      Toast.success('Account created and signed in!');
    } catch (e) {
      Toast.error(e.message);
    }
  };

  return (
```

```
      <div className="space-y-6">
        <h3 className="text-xl font-medium text-gray-900 text-center">Create a New
Account</h3>
        <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
          <Input label="First Name" id="signup-fname" {...register('firstName')}
disabled={isSubmitting} />
          {errors.firstName && <p className="text-red-500 text-
sm">{errors.firstName.message}</p>}

          <Input label="Last Name" id="signup-lname" {...register('lastName')}
disabled={isSubmitting} />
          {errors.lastName && <p className="text-red-500 text-
sm">{errors.lastName.message}</p>}

          <Input label="Email Address" id="signup-email" {...register('email')}
disabled={isSubmitting} />
          {errors.email && <p className="text-red-500 text-sm">{errors.email.message}</p>}

          <Input label="Password" id="signup-password" type="password"
{...register('password')} disabled={isSubmitting} />
          {errors.password && <p className="text-red-500 text-
sm">{errors.password.message}</p>}

          <Button type="submit" className="w-full" disabled={isSubmitting}>
            {isSubmitting ? 'Creating...' : 'Create Account'}
          </Button>
        </form>
        <div className="text-sm text-center">
          Already have an account?{' '}
          <button onClick={() => setAuthPage('login')} className="font-medium text-indigo-600
hover:text-indigo-500">
            Sign In
          </button>
        </div>
      </div>
    );
};

const AuthPage = () => {
    const { page } = useAuth();
    const authPath = page.split('/')[2];
    const [authPage, setAuthPage] = useState(authPath || 'login');

    useEffect(() => {
```

```
        if (authPath && authPath !== authPage) setAuthPage(authPath);
    }, [authPath, authPage]);

    const handleResetSuccess = () => setAuthPage('login');

    return (
        <AuthLayout>
            {authPage === 'login' && <LoginForm setAuthPage={setAuthPage} />}
            {authPage === 'signup' && <SignUpForm setAuthPage={setAuthPage} />}
            {authPage === 'reset-request' && <RequestResetForm
onResetSuccess={handleResetSuccess} />}
        </AuthLayout>
    );
};

// --- CREDENTIALS PAGES ---

const CredentialForm = ({ credential, onSave, onCancel }) => {
    const { api } = useAuth();
    const isEdit = !!credential;
    const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
        resolver: zodResolver(CredentialSchema),
        defaultValues: credential ? { name: credential.name, type: credential.type, encryptedValue:
'**********' } : {} // Use mask for existing value
    });

    const onSubmit = async (data) => {
        try {
            if (isEdit) {
                // In a real app, we'd update here (preserving masked value if unchanged)
                Toast.success('Credential updated (Mock).');
            } else {
                await api.createCredential(data);
            }
            onSave();
        } catch (e) {
            Toast.error(`Operation failed: ${e.message}`);
        }
    };

    return (
        <Card title={isEdit ? 'Edit Credential' : 'Create New Credential'} className="max-w-xl mx-
auto">
            <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
```

```
            <Input label="Credential Name" id="cred-name" {...register('name')}
disabled={isSubmitting} />
            {errors.name && <p className="text-red-500 text-sm">{errors.name.message}</p>}

            <Input label="Credential Type (e.g., API Key, DB Secret)" id="cred-type"
{...register('type')} disabled={isSubmitting} />
            {errors.type && <p className="text-red-500 text-sm">{errors.type.message}</p>}

            <Textarea label="Encrypted Value (Mock)" id="cred-value"
{...register('encryptedValue')} disabled={isSubmitting} className="font-mono" />
            {errors.encryptedValue && <p className="text-red-500 text-
sm">{errors.encryptedValue.message}</p>}

            <div className="flex justify-end space-x-3 pt-2">
               <Button type="button" variant="secondary" onClick={onCancel}
disabled={isSubmitting}>Cancel</Button>
               <Button type="submit" disabled={isSubmitting}>
                  {isSubmitting ? 'Saving...' : 'Save Credential'}
               </Button>
            </div>
         </form>
      </Card>
   );
};

const CredentialManagementPage = () => {
   const { api, can } = useAuth();
   const [credentials, setCredentials] = useState([]);
   const [isLoading, setIsLoading] = useState(true);
   const [isFormOpen, setIsFormOpen] = useState(false);

   const fetchCredentials = useCallback(async () => {
      if (!can('credential:manage')) return setIsLoading(false);
      setIsLoading(true);
      try {
         const data = await api.getCredentials();
         setCredentials(data);
      } catch (e) {
         Toast.error(`Failed to load credentials: ${e.message}`);
         setCredentials([]);
      } finally {
         setIsLoading(false);
      }
   }, [api, can]);
```

```
  useEffect(() => {
    fetchCredentials();
  }, [fetchCredentials]);

  if (!can('credential:manage')) {
    return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-red-500">You do not have permission to manage credentials.</p></Card>;
  }

  if (isFormOpen) {
    return <CredentialForm onSave={() => { setIsFormOpen(false); fetchCredentials(); }} onCancel={() => setIsFormOpen(false)} />;
  }

  const handleDelete = async (id) => {
    if (window.confirm('Are you sure you want to delete this credential?')) {
      try {
        await api.deleteCredential(id);
        fetchCredentials();
      } catch (e) {
        Toast.error(`Delete failed: ${e.message}`);
      }
    }
  };

  const columns = [
    { header: 'Name', cell: (c) => c.name },
    { header: 'Type', cell: (c) => <Badge variant="secondary">{c.type}</Badge> },
    { header: 'Created By', cell: (c) => c.created_by },
    { header: 'Actions', cell: (c) => (
      <DropdownMenu trigger={<Button size="sm" variant="secondary">Actions</Button>}>
        <DropdownMenuItem onClick={() => Toast.info(`Editing credential ${c.credentialId}`)}>Edit</DropdownMenuItem>
        <DropdownMenuItem onClick={() => handleDelete(c.credentialId)} className="text-red-600">Delete</DropdownMenuItem>
      </DropdownMenu>
    )},
  ];

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <h1 className="text-3xl font-bold text-gray-900">Credential Management</h1>
```

```
            <Button onClick={() => setIsFormOpen(true)}>+ Create Credential</Button>
          </div>
          <Card>
            {isLoading ? (
              <p className="text-center text-gray-500">Loading credentials...</p>
            ) : (
              <DataTable columns={columns} data={credentials} />
            )}
          </Card>
        </div>
    );
};

// --- JOB PAGES ---

const JobForm = ({ job, onSave, onCancel }) => {
    const { api, can } = useAuth();
    const isEdit = !!job;
    const [credentialOptions, setCredentialOptions] = useState([]);
    const { register, handleSubmit, formState: { errors, isSubmitting }, watch, reset } = useForm({
        resolver: zodResolver(JobCreateSchema),
        defaultValues: {
            jobTypeId: job?.jobTypeId || 1,
            name: job?.name || '',
            description: job?.description || '',
            priority: job?.priority || 2,
            timeout: job?.timeout || 60,
            retryAttempts: job?.retryAttempts || 0,
            retryDelay: job?.retryDelay || 5,
            isactive: job?.isactive ?? true,
            authMechanism: job?.authMechanism || 'Runner Service Account',
            credentialId: job?.credentialId || null,
            executableConfig: job?.executableConfig || {},
            apiConfig: job?.apiConfig || { httpMethod: 'GET', authType: 'None' },
            scriptConfig: job?.scriptConfig || {},
            aiConfig: job?.aiConfig || {},
        }
    });

    const jobTypeId = watch('jobTypeId');
    const authMechanism = watch('authMechanism');

    // Fetch credentials for selector (Slice 17.0)
    useEffect(() => {
```

```
        if (can('credential:manage')) {
            api.getCredentialOptions().then(setCredentialOptions);
        }
    }, [api, can]);

    const onSubmit = async (data) => {
        try {
            // Filter config based on job type before sending
            const jobData = { ...data };
            jobData.executableConfig = data.jobTypeId === 1 ? data.executableConfig : undefined;
            jobData.apiConfig = data.jobTypeId === 2 ? data.apiConfig : undefined;
            jobData.scriptConfig = data.jobTypeId === 4 ? data.scriptConfig : undefined;
            jobData.aiConfig = data.jobTypeId === 5 ? data.aiConfig : undefined;

            // Set credentialId based on authMechanism
            if (jobData.authMechanism === 'Runner Service Account') {
                jobData.credentialId = null;
            } else if (jobData.authMechanism === 'Specific Credential') {
                if (!jobData.credentialId) throw new Error('Specific Credential selected, but none
provided.');
                jobData.credentialId = Number(jobData.credentialId);
            }


            if (isEdit) {
                await api.updateJob(job.jobId, jobData);
            } else {
                await api.createJob(jobData);
            }
            onSave();
        } catch (e) {
            Toast.error(`Job operation failed: ${e.message}`);
            console.error(e);
        }
    };

    const requiresCredentialSelector = jobTypeId === 2 || jobTypeId === 4 || jobTypeId === 5;

    return (
        <Card title={isEdit ? `Edit Job: ${job.name}` : 'Create New Job'} className="max-w-4xl
mx-auto">
            <form onSubmit={handleSubmit(onSubmit)} className="space-y-6">
                {/* Section 1: General */}
                <div className="space-y-4">
```

```
                <h3 className="text-lg font-semibold border-b pb-2">General Details</h3>
                <div className="grid grid-cols-2 gap-4">
                  <Input label="Job Name" id="name" {...register('name')} />
                  {errors.name && <p className="text-red-500 text-sm col-span-
2">{errors.name.message}</p>}

                  <Select label="Job Type" id="jobTypeId" options={JOB_TYPES.map(t => ({
value: t.id, label: t.label }))} {...register('jobTypeId', { valueAsNumber: true })} className="col-
span-1" />
                  {errors.jobTypeId && <p className="text-red-500 text-sm col-span-
2">{errors.jobTypeId.message}</p>}

                  <Textarea label="Description" id="description" {...register('description')}
className="col-span-2" />
                </div>
              </div>

              <Separator />

              {/* Section 2: Execution Configuration (Conditional on Type) */}
              <div className="space-y-4">
                <h3 className="text-lg font-semibold border-b pb-2">Execution
Configuration</h3>

                {/* Type 1: Executable */}
                {jobTypeId === 1 && (
                  <div className="grid grid-cols-2 gap-4">
                    <Input label="Executable Path" id="executableConfig.executable"
{...register('executableConfig.executable')} className="col-span-2" />
                    {errors.executableConfig?.executable && <p className="text-red-500 text-sm
col-span-2">{errors.executableConfig.executable.message}</p>}
                    <Textarea label="Parameters" id="executableConfig.parameters"
{...register('executableConfig.parameters')} className="col-span-2 font-mono" />
                    <Input label="Working Directory" id="executableConfig.workingDirectory"
{...register('executableConfig.workingDirectory')} className="col-span-2" />
                  </div>
                )}

                {/* Type 2: API Call (Slice 8.0) */}
                {jobTypeId === 2 && (
                  <div className="grid grid-cols-2 gap-4">
                    <Input label="Target URL" id="apiConfig.url" {...register('apiConfig.url')}
className="col-span-2" />
```

```
                    <Select label="HTTP Method" id="apiConfig.httpMethod" options={['GET',
'POST', 'PUT', 'DELETE', 'PATCH'].map(m => ({ value: m, label: m }))}
{...register('apiConfig.httpMethod')} />
                    <Select label="Auth Type" id="apiConfig.authType" options={['None', 'Bearer
Token', 'Basic Auth'].map(m => ({ value: m, label: m }))} {...register('apiConfig.authType')} />
                    <Textarea label="Headers (JSON)" id="apiConfig.headers"
{...register('apiConfig.headers')} className="col-span-2 font-mono" />
                    <Textarea label="Request Body" id="apiConfig.requestBody"
{...register('apiConfig.requestBody')} className="col-span-2 font-mono" />
                </div>
            )}

            {/* Type 4: Python Script (Slice 16.0) */}
            {jobTypeId === 4 && (
                <div className="grid grid-cols-2 gap-4">
                    <Textarea label="Python Script Content" id="scriptConfig.scriptContent"
{...register('scriptConfig.scriptContent')} className="col-span-2 font-mono min-h-[200px]" />
                    {errors.scriptConfig?.scriptContent && <p className="text-red-500 text-sm
col-span-2">{errors.scriptConfig.scriptContent.message}</p>}
                    <Input label="Runtime Environment (e.g., python3-env)"
id="scriptConfig.runtimeEnvironment" {...register('scriptConfig.runtimeEnvironment')}
className="col-span-2" />
                </div>
            )}

            {/* Type 5: AI Agent (Slice 16.0) */}
            {jobTypeId === 5 && (
                <div className="grid grid-cols-2 gap-4">
                    <Input label="Model ID (e.g., gemini-2.5-flash)" id="aiConfig.modelId"
{...register('aiConfig.modelId')} className="col-span-2" />
                    {errors.aiConfig?.modelId && <p className="text-red-500 text-sm col-span-
2">{errors.aiConfig.modelId.message}</p>}
                    <Textarea label="Agent Configuration (JSON)" id="aiConfig.agentConfig"
{...register('aiConfig.agentConfig')} className="col-span-2 font-mono" />
                    <Input label="Runtime Environment" id="aiConfig.runtimeEnvironment"
{...register('aiConfig.runtimeEnvironment')} className="col-span-2" />
                </div>
            )}
        </div>

        <Separator />

        {/* Section 3: Credentials (Slice 17.0) */}
        {(requiresCredentialSelector && can('credential:manage')) && (
```

```
                <div className="space-y-4">
                    <h3 className="text-lg font-semibold border-b pb-2">Credential / Authentication</h3>
                    <p className="text-sm text-gray-500">How should the runner authenticate for this job?</p>
                    <div className="grid grid-cols-2 gap-4">
                        <Select
                            label="Authentication Mechanism"
                            id="authMechanism"
                            options={[{ value: 'Runner Service Account', label: 'Use Runner Service Account (Service Account Auth)' }, { value: 'Specific Credential', label: 'Use Specific Credential (API Key/Secret Injection)' }]}
                            {...register('authMechanism')}
                        />
                        {authMechanism === 'Specific Credential' && (
                            <Select
                                label="Select Credential"
                                id="credentialId"
                                options={[{ value: '', label: 'Select a credential', disabled: true }, ...credentialOptions]}
                                {...register('credentialId', { valueAsNumber: true })}
                            />
                        )}
                    </div>
                    {authMechanism === 'Specific Credential' && errors.credentialId && <p className="text-red-500 text-sm col-span-2">{errors.credentialId.message}</p>}
                </div>
            )}
            {/* Fallback if user doesn't have credential:manage but job requires it */}
            {(requiresCredentialSelector && !can('credential:manage')) && (
                <div className="p-4 bg-yellow-50 border border-yellow-200 rounded-lg text-sm text-yellow-800">
                    <MessageSquare className="h-5 w-5 inline mr-2" />
                    This job type requires credentials. It will run with the default Runner Service Account, as you lack permission to select a specific credential.
                </div>
            )}

            <Separator />

            {/* Section 4: Configuration */}
            <div className="space-y-4">
                <h3 className="text-lg font-semibold border-b pb-2">Runtime Configuration</h3>
                <div className="grid grid-cols-3 gap-4">
```

```
                    <Input label="Timeout (sec)" id="timeout" type="number" {...register('timeout', {
valueAsNumber: true })} />
                    <Input label="Retry Attempts" id="retryAttempts" type="number"
{...register('retryAttempts', { valueAsNumber: true })} />
                    <Input label="Retry Delay (sec)" id="retryDelay" type="number"
{...register('retryDelay', { valueAsNumber: true })} />
                    <Select label="Priority" id="priority" options={PRIORITY_OPTIONS}
{...register('priority', { valueAsNumber: true })} />
                    <div className="flex items-end pt-4 col-span-2">
                        <Switch id="isactive" label="Job Definition is Active"
checked={watch('isactive')} onCheckedChange={(val) => reset({ ...watch(), isactive: val })} />
                    </div>
                </div>
                {errors.root && <p className="text-red-500 text-sm col-span-
3">{errors.root.message}</p>}
            </div>

            <div className="flex justify-end space-x-3 pt-4">
                <Button type="button" variant="secondary" onClick={onCancel}
disabled={isSubmitting}>Cancel</Button>
                <Button type="submit" disabled={isSubmitting}>
                    {isSubmitting ? 'Saving...' : 'Save Job'}
                </Button>
            </div>
        </form>
    </Card>
  );
};

const JobPage = () => {
  const { api, setPage, can } = useAuth();
  const pathParts = useAuth().page.split('/');
  const id = pathParts[pathParts.length - 1];
  const isCreateMode = id === 'new';

  const [job, setJob] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  const fetchJob = useCallback(async () => {
    if (isCreateMode) {
      setIsLoading(false);
      return;
    }
    setIsLoading(true);
```

```
        try {
            const data = await api.getJob(id);
            setJob(data);
        } catch (e) {
            Toast.error(e.message);
        } finally {
            setIsLoading(false);
        }
    }, [api, id, isCreateMode]);

    useEffect(() => {
        fetchJob();
    }, [fetchJob]);

    if (!can('job:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to manage job definitions.</p></Card>;
    }

    if (!isCreateMode && isLoading) {
        return <Card title="Loading..." className="max-w-4xl mx-auto"><p>Fetching job
details...</p></Card>;
    }

    if (!isCreateMode && !job) {
        return <Card title="Not Found" className="max-w-4xl mx-auto"><p>The requested job
does not exist or access is denied.</p></Card>;
    }

    const handleSave = () => {
        setPage('/dashboard/jobs');
    };

    const handleCancel = () => {
        setPage('/dashboard/jobs');
    };

    return <JobForm job={job} onSave={handleSave} onCancel={handleCancel} />;
};

const JobListPage = () => {
    const { api, setPage, can } = useAuth();
    const [jobs, setJobs] = useState([]);
    const [isLoading, setIsLoading] = useState(true);
```

```
    const [filterQuery, setFilterQuery] = useState('');
    const deferredQuery = useDeferredValue(filterQuery);
    const [testJob, setTestJob] = useState(null);
    const [testResult, setTestResult] = useState(null);

    const fetchJobs = useCallback(async (query) => {
        setIsLoading(true);
        try {
            const data = await api.getJobs(query);
            setJobs(data);
        } catch (e) {
            Toast.error(`Failed to load jobs: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        fetchJobs(deferredQuery);
    }, [deferredQuery, fetchJobs]);

    if (!can('job:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to view this list.</p></Card>;
    }

    const handleDelete = async (id) => {
        if (window.confirm('Are you sure you want to delete this job? This will also remove any
scheduled links.')) {
            try {
                await api.deleteJob(id);
                fetchJobs(filterQuery);
            } catch (e) {
                Toast.error(`Delete failed: ${e.message}`);
            }
        }
    };

    const handleTestRun = async (job) => {
        setTestJob(job);
        setTestResult('running');
        try {
            const result = await api.testJob(job.jobId);
            setTestResult(result);
```

```
      } catch (e) {
        setTestResult({ executionStatus: 'failed', errorOutput: e.message, output: 'Test could not be initiated.' });
        Toast.error(`Test run failed: ${e.message}`);
      }
    };

    const columns = [
      { header: 'Name', cell: (job) => <div className="font-semibold">{job.name} <p className="text-xs text-gray-500 truncate">{job.description}</p></div> },
      { header: 'Type', cell: (job) => <Badge variant="secondary">{job.jobTypeLabel}</Badge> },
      { header: 'Auth', cell: (job) => job.credentialId ? <Badge variant="info">{job.credentialName}</Badge> : <Badge variant="secondary">Service Account</Badge> },
      { header: 'Active', cell: (job) => <Badge variant={job.isactive ? 'success' : 'secondary'}>{job.isactive ? 'Yes' : 'No'}</Badge> },
      { header: 'Created By', cell: (job) => job.createdby },
      {
        header: 'Actions',
        cell: (job) => (
          <DropdownMenu trigger={<Button size="sm" variant="secondary">Actions</Button>}>
            {can('job:create_edit') && <DropdownMenuItem onClick={() => setPage(`/dashboard/jobs/${job.jobId}`)}>Edit</DropdownMenuItem>}
            {can('job:test_sync') && <DropdownMenuItem onClick={() => handleTestRun(job)}>Test Run</DropdownMenuItem>}
            {can('job:create_edit') && <DropdownMenuItem onClick={() => handleDelete(job.jobId)} className="text-red-600">Delete</DropdownMenuItem>}
          </DropdownMenu>
        ),
      },
    ];

    return (
      <div className="space-y-6">
        <div className="flex justify-between items-center">
          <h1 className="text-3xl font-bold text-gray-900">Job Definitions</h1>
          {can('job:create_edit') && (
            <Button onClick={() => setPage('/dashboard/jobs/new')}>+ Create Job</Button>
          )}
        </div>
        <Input
          placeholder="Search jobs by name or description..."
          value={filterQuery}
```

```
            onChange={(e) => setFilterQuery(e.target.value)}
          />
          <Card>
            {isLoading ? (
              <p className="text-center text-gray-500">Loading jobs...</p>
            ) : (
              <DataTable columns={columns} data={jobs} />
            )}
          </Card>
          <TestJobDialog job={testJob} result={testResult} onClose={() => { setTestJob(null);
setTestResult(null); }} />
        </div>
    );
};

const TestJobDialog = ({ job, result, onClose }) => {
    const isRunning = result === 'running';

    const getStatusVariant = (status) => {
        if (status === 'completed') return 'success';
        if (status === 'failed') return 'critical';
        return 'info';
    };

    if (!job) return null;

    return (
        <Dialog title={`Test Run: ${job.name}`} open={!!job} onClose={!isRunning ? onClose : () =>
{}} className="max-w-3xl">
            {isRunning && (
                <div className="text-center p-8">
                    <Code className="h-10 w-10 mx-auto text-indigo-500 animate-pulse" />
                    <p className="mt-4 text-lg font-semibold">Running synchronous test...</p>
                    <p className="text-sm text-gray-500">Please wait. Results will appear
automatically.</p>
                </div>
            )}
            {!isRunning && result && result.executionStatus && (
                <div className="space-y-6">
                    <div className="flex items-center space-x-4 p-3 border rounded-lg">
                        <Badge variant={getStatusVariant(result.executionStatus)} className="text-
base py-1 px-3">
                            {result.executionStatus.toUpperCase()}
                        </Badge>
```

```
                    <div className="text-sm">
                        <p className="font-medium">Exit Code: {result.exitCode}</p>
                        <p className="text-gray-500">Duration: {formatDuration(result.duration)}</p>
                    </div>
                </div>

                <h4 className="text-lg font-semibold">Output Logs</h4>
                <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
                    <Card title="Standard Output (stdout)" className="p-0 border">
                        <pre className="text-xs p-3 overflow-auto bg-gray-50 max-h-48 whitespace-
pre-wrap">{result.output || 'No stdout log.'}</pre>
                    </Card>
                    <Card title="Standard Error (stderr)" className="p-0 border">
                        <pre className="text-xs p-3 overflow-auto bg-gray-50 max-h-48 whitespace-
pre-wrap text-red-700">{result.errorOutput || 'No stderr log.'}</pre>
                    </Card>
                </div>

                <div className="flex justify-end">
                    <Button onClick={onClose}>Close</Button>
                </div>
            </div>
        )}
    </Dialog>
  );
};

// --- SCHEDULE PAGES ---

const ScheduleForm = ({ schedule, onSave, onCancel }) => {
  const { api } = useAuth();
  const isEdit = !!schedule;
  const { register, handleSubmit, formState: { errors, isSubmitting }, reset, watch } = useForm({
    resolver: zodResolver(ScheduleCreateSchema),
    defaultValues: {
      name: schedule?.name || '',
      description: schedule?.description || '',
      pattern: schedule?.pattern || 1,
      starttime: schedule?.starttime || '09:00',
      timezone: schedule?.timezone || 'UTC',
      weekdays: schedule?.weekdays || '1,2,3,4,5',
      dayofmonth: schedule?.dayofmonth || 1,
      weeknumber: schedule?.weeknumber || 1,
      pattern_string: schedule?.pattern_string || '0 * * * *',
```

```
            isactive: schedule?.isactive ?? true,
        }
    });

    const pattern = watch('pattern');
    const weekdays = watch('weekdays');
    const resetWeekdays = (newDays) => reset({ ...watch(), weekdays: newDays.join(',') });

    const onSubmit = async (data) => {
        try {
            if (data.pattern === 4) {
                // For Custom Cron, strip non-cron fields
                data.starttime = undefined;
                data.timezone = undefined;
                data.weekdays = undefined;
                data.dayofmonth = undefined;
                data.weeknumber = undefined;
            }

            if (isEdit) {
                await api.updateSchedule(schedule.scheduleId, data);
            } else {
                await api.createSchedule(data);
            }
            onSave();
        } catch (e) {
            Toast.error(`Schedule operation failed: ${e.message}`);
        }
    };

    const isSimpleTime = pattern === 1 || pattern === 2 || pattern === 3;
    const isCustomCron = pattern === 4;

    return (
        <Card title={isEdit ? `Edit Schedule: ${schedule.name}` : 'Create New Schedule'}
className="max-w-3xl mx-auto">
            <form onSubmit={handleSubmit(onSubmit)} className="space-y-6">
                {/* Section 1: General */}
                <div className="space-y-4">
                    <h3 className="text-lg font-semibold border-b pb-2">General Details</h3>
                    <div className="grid grid-cols-2 gap-4">
                        <Input label="Schedule Name" id="name" {...register('name')} />
                        {errors.name && <p className="text-red-500 text-sm col-span-
2">{errors.name.message}</p>}
```

```
                    <Select label="Pattern Type" id="pattern"
options={SCHEDULE_PATTERN_OPTIONS} {...register('pattern', { valueAsNumber: true })}
className="col-span-1" />
                    {errors.pattern && <p className="text-red-500 text-sm col-span-
2">{errors.pattern.message}</p>}

                    <div className="flex items-end pt-4">
                      <Switch id="isactive" label="Schedule is Active" checked={watch('isactive')}
onCheckedChange={(val) => reset({ ...watch(), isactive: val })} />
                    </div>
                    <Textarea label="Description" id="description" {...register('description')}
className="col-span-2" />
                  </div>
                  {errors.root && <p className="text-red-500 text-sm">{errors.root.message}</p>}
                </div>

                <Separator />

                {/* Section 2: Timing / Pattern Configuration */}
                <div className="space-y-4">
                  <h3 className="text-lg font-semibold border-b pb-2">Timing Configuration</h3>
                  {errors.starttime && <p className="text-red-500 text-sm col-span-2">Time and/or
Pattern configuration is invalid.</p>}

                  {/* Simple Time (Daily, Weekly, Monthly) */}
                  {isSimpleTime && (
                    <div className="grid grid-cols-2 gap-4">
                      <Input label="Run Time (HH:MM)" id="starttime" type="time"
{...register('starttime')} />
                      {errors.starttime && <p className="text-red-500 text-sm col-span-
2">{errors.starttime.message}</p>}
                      <Select label="Timezone" id="timezone" options={[{ value: 'UTC', label: 'UTC'
}, { value: 'America/New_York', label: 'EST/EDT' }]} {...register('timezone')} />
                    </div>
                  )}

                  {/* Pattern 2: Weekly (requires weekdays) */}
                  {(pattern === 2) && (
                    <div className="pt-2">
                      <label className="text-sm font-medium text-gray-700 block mb-2">Days of
the Week</label>
                      <div className="flex flex-wrap gap-2">
                        {WEEKDAY_OPTIONS.map(day => (
```

```
                        <Button
                            key={day.value}
                            type="button"
                            variant={weekdays?.split(',').includes(String(day.value)) ? 'primary' :
'secondary'}
                            onClick={() => {
                                const current = weekdays?.split(',').map(Number) || [];
                                const index = current.indexOf(day.value);
                                let newDays;
                                if (index > -1) {
                                    newDays = current.filter(d => d !== day.value);
                                } else {
                                    newDays = [...current, day.value];
                                }
                                resetWeekdays(newDays);
                            }}
                            size="sm"
                        >
                            {day.label}
                        </Button>
                    ))}
                </div>
                {errors.weekdays && <p className="text-red-500 text-sm mt-
2">{errors.weekdays.message}</p>}
            </div>
        )}

        {/* Pattern 3: Monthly (requires day or week/day combo) */}
        {(pattern === 3) && (
            <div className="pt-2 space-y-3">
                <p className="text-sm font-medium text-gray-700 block">Monthly recurrence
must specify day of month or a week/day combination.</p>
                <Input label="Day of Month (1-31)" id="dayofmonth" type="number"
{...register('dayofmonth', { valueAsNumber: true })} min="1" max="31" />
                {errors.dayofmonth && <p className="text-red-500 text-
sm">{errors.dayofmonth.message}</p>}
            </div>
        )}

        {/* Pattern 4: Custom Cron (Slice 16.1) */}
        {isCustomCron && (
            <div className="pt-2 space-y-3">
                <p className="text-sm font-medium text-gray-700 block">Enter a 5-field
Cron Expression (Minute Hour DayOfMonth Month DayOfWeek)</p>
```

```
                        <Textarea label="Cron Expression" id="pattern_string"
{...register('pattern_string')} className="font-mono" />
                        {errors.pattern_string && <p className="text-red-500 text-sm mt-
2">{errors.pattern_string.message}</p>}
                        {validateCronStructure(watch('pattern_string')) && <p className="text-red-
500 text-sm">{validateCronStructure(watch('pattern_string'))}</p>}
                    </div>
                )}
            </div>

            <div className="flex justify-end space-x-3 pt-4">
                <Button type="button" variant="secondary" onClick={onCancel}
disabled={isSubmitting}>Cancel</Button>
                <Button type="submit" disabled={isSubmitting}>
                    {isSubmitting ? 'Saving...' : 'Save Schedule'}
                </Button>
            </div>
        </form>
    </Card>
  );
};

const SchedulePage = () => {
  const { api, setPage, can } = useAuth();
  const pathParts = useAuth().page.split('/');
  const id = pathParts[pathParts.length - 1];
  const isCreateMode = id === 'new';

  const [schedule, setSchedule] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  const fetchSchedule = useCallback(async () => {
    if (isCreateMode) {
      setIsLoading(false);
      return;
    }
    setIsLoading(true);
    try {
      const data = await api.getSchedule(id);
      setSchedule(data);
    } catch (e) {
      Toast.error(e.message);
    } finally {
      setIsLoading(false);
```

```
        }
    }, [api, id, isCreateMode]);

    useEffect(() => {
        fetchSchedule();
    }, [fetchSchedule]);

    if (!can('job:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-red-500">You do not have the permission to manage schedules.</p></Card>;
    }

    if (!isCreateMode && isLoading) {
        return <Card title="Loading..." className="max-w-3xl mx-auto"><p>Fetching schedule details...</p></Card>;
    }

    if (!isCreateMode && !schedule) {
        return <Card title="Not Found" className="max-w-3xl mx-auto"><p>The requested schedule does not exist or access is denied.</p></Card>;
    }

    const handleSave = () => {
        setPage('/dashboard/schedules');
    };

    const handleCancel = () => {
        setPage('/dashboard/schedules');
    };

    return <ScheduleForm schedule={schedule} onSave={handleSave} onCancel={handleCancel} />;
};

const ScheduleListPage = () => {
    const { api, setPage, can } = useAuth();
    const [schedules, setSchedules] = useState([]);
    const [isLoading, setIsLoading] = useState(true);
    const [filterQuery, setFilterQuery] = useState('');
    const deferredQuery = useDeferredValue(filterQuery);

    const fetchSchedules = useCallback(async (query) => {
        setIsLoading(true);
        try {
```

```
            const data = await api.getSchedules(query);
            setSchedules(data);
        } catch (e) {
            Toast.error(`Failed to load schedules: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        fetchSchedules(deferredQuery);
    }, [deferredQuery, fetchSchedules]);

    if (!can('job:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to view this list.</p></Card>;
    }

    const handleDelete = async (id) => {
        if (window.confirm('Are you sure you want to delete this schedule? This will also remove
any scheduled links.')) {
            try {
                await api.deleteSchedule(id);
                fetchSchedules(filterQuery);
            } catch (e) {
                Toast.error(`Delete failed: ${e.message}`);
            }
        }
    };

    const columns = [
        { header: 'Name', cell: (s) => s.name },
        { header: 'Schedule', cell: (s) => s.patternLabel },
        { header: 'Active', cell: (s) => <Badge variant={s.isactive ? 'success' :
'secondary'}>{s.isactive ? 'Yes' : 'No'}</Badge> },
        { header: 'Created By', cell: (s) => s.createdby },
        {
            header: 'Actions',
            cell: (s) => (
                <DropdownMenu trigger={<Button size="sm"
variant="secondary">Actions</Button>}>
                    {can('job:create_edit') && <DropdownMenuItem onClick={() =>
setPage(`/dashboard/schedules/${s.scheduleId}`)}>Edit</DropdownMenuItem>}
```

```
                {can('job:create_edit') && <DropdownMenuItem onClick={() =>
handleDelete(s.scheduleId)} className="text-red-600">Delete</DropdownMenuItem>}
            </DropdownMenu>
        ),
    },
  ];

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <h1 className="text-3xl font-bold text-gray-900">Schedules</h1>
        {can('job:create_edit') && (
          <Button onClick={() => setPage('/dashboard/schedules/new')}>+ Create
Schedule</Button>
        )}
      </div>
      <Input
        placeholder="Search schedules by name or pattern..."
        value={filterQuery}
        onChange={(e) => setFilterQuery(e.target.value)}
      />
      <Card>
        {isLoading ? (
          <p className="text-center text-gray-500">Loading schedules...</p>
        ) : (
          <DataTable columns={columns} data={schedules} />
        )}
      </Card>
    </div>
  );
};

// --- SCHEDULED JOB PAGES ---

const ScheduledJobForm = ({ scheduledJob, onSave, onCancel }) => {
  const { api } = useAuth();
  const isEdit = !!scheduledJob;
  const [jobs, setJobs] = useState([]);
  const [schedules, setSchedules] = useState([]);
  const { register, handleSubmit, formState: { errors, isSubmitting }, watch } = useForm({
    resolver: zodResolver(ScheduledJobCreateSchema),
    defaultValues: {
      jobId: scheduledJob?.jobId || '',
      scheduleId: scheduledJob?.scheduleId || '',
```

```
            name: scheduledJob?.name || '',
            description: scheduledJob?.description || '',
            priority: scheduledJob?.priority || 2,
            maxConcurrentRuns: scheduledJob?.maxConcurrentRuns || 1,
            queueTimeout: scheduledJob?.queueTimeout || 60,
            isactive: scheduledJob?.isactive ?? true,
            alertOnFailure: scheduledJob?.alertOnFailure ?? true,
            alertOnSuccess: scheduledJob?.alertOnSuccess ?? false,
            alertOnDelay: scheduledJob?.alertOnDelay ?? true,
            delayThreshold: scheduledJob?.delayThreshold || 15,
            escalationEmail: scheduledJob?.escalationEmail || '',
        }
    });

    const alertOnDelay = watch('alertOnDelay');

    useEffect(() => {
        // Fetch data for dropdowns
        Promise.all([api.getJobs(), api.getSchedules()])
            .then(([jobsData, schedulesData]) => {
                setJobs(jobsData.map(j => ({ value: j.jobId, label: j.name })));
                setSchedules(schedulesData.map(s => ({ value: s.scheduleId, label: `${s.name}
(${s.patternLabel})` })));
            })
            .catch(e => Toast.error(`Failed to load link data: ${e.message}`));
    }, [api]);


    const onSubmit = async (data) => {
        try {
            if (isEdit) {
                await api.updateScheduledJob(scheduledJob.scheduledJobId, data);
            } else {
                await api.createScheduledJob({ ...data, jobId: Number(data.jobId), scheduleId:
Number(data.scheduleId) });
            }
            onSave();
        } catch (e) {
            Toast.error(`Link operation failed: ${e.message}`);
            console.error(e);
        }
    };

    return (
```

```
      <Card title={isEdit ? `Edit Link: ${scheduledJob.name}` : 'Link Job to Schedule'}
className="max-w-3xl mx-auto">
        <form onSubmit={handleSubmit(onSubmit)} className="space-y-6">
          {/* Section 1: Link */}
          <div className="space-y-4">
            <h3 className="text-lg font-semibold border-b pb-2">Job & Schedule
Selection</h3>
            <div className="grid grid-cols-2 gap-4">
              <Select label="Job Definition" id="jobId" options={[{ value: '', label: 'Select Job',
disabled: true }, ...jobs]} {...register('jobId')} disabled={isEdit} />
              {errors.jobId && <p className="text-red-500 text-sm col-span-
2">{errors.jobId.message}</p>}
              <Select label="Schedule" id="scheduleId" options={[{ value: '', label: 'Select
Schedule', disabled: true }, ...schedules]} {...register('scheduleId')} disabled={isEdit} />
              {errors.scheduleId && <p className="text-red-500 text-sm col-span-
2">{errors.scheduleId.message}</p>}

              <Input label="Link Name" id="name" {...register('name')} className="col-span-
2" />
              {errors.name && <p className="text-red-500 text-sm col-span-
2">{errors.name.message}</p>}
              <Textarea label="Description" id="description" {...register('description')}
className="col-span-2" />
            </div>
          </div>

          <Separator />

          {/* Section 2: Configuration */}
          <div className="space-y-4">
            <h3 className="text-lg font-semibold border-b pb-2">Configuration</h3>
            <div className="grid grid-cols-3 gap-4">
              <Input label="Max Concurrent Runs" id="maxConcurrentRuns" type="number"
{...register('maxConcurrentRuns', { valueAsNumber: true })} />
              <Input label="Queue Timeout (sec)" id="queueTimeout" type="number"
{...register('queueTimeout', { valueAsNumber: true })} />
              <Select label="Priority Override" id="priority" options={PRIORITY_OPTIONS}
{...register('priority', { valueAsNumber: true })} />
              <div className="flex items-end pt-4 col-span-3">
                <Switch id="isactive" label="Link is Active" checked={watch('isactive')}
onCheckedChange={(val) => reset({ ...watch(), isactive: val })} />
              </div>
            </div>
          </div>
```

```
            <Separator />

            {/* Section 3: Alerting */}
            <div className="space-y-4">
               <h3 className="text-lg font-semibold border-b pb-2">Alerting</h3>
               <div className="grid grid-cols-2 gap-4">
                  <Checkbox label="Alert on Failure" id="alertOnFailure"
{...register('alertOnFailure')} />
                  <Checkbox label="Alert on Success" id="alertOnSuccess"
{...register('alertOnSuccess')} />
                  <Checkbox label="Alert on Delay" id="alertOnDelay" {...register('alertOnDelay')}
/>
                  <Input label="Escalation Email (Group)" id="escalationEmail"
{...register('escalationEmail')} placeholder="e.g., ops-team@example.com" />
               </div>
               {alertOnDelay && (
                  <div className="grid grid-cols-2 gap-4 pt-2">
                     <Input label="Delay Threshold (min)" id="delayThreshold" type="number"
{...register('delayThreshold', { valueAsNumber: true })} />
                  </div>
               )}
            </div>

            <div className="flex justify-end space-x-3 pt-4">
               <Button type="button" variant="secondary" onClick={onCancel}
disabled={isSubmitting}>Cancel</Button>
               <Button type="submit" disabled={isSubmitting}>
                  {isSubmitting ? 'Saving Link...' : isEdit ? 'Save Changes' : 'Create Link'}
               </Button>
            </div>
         </form>
      </Card>
   );
};

const ScheduledJobPage = () => {
   const { api, setPage, can } = useAuth();
   const pathParts = useAuth().page.split('/');
   const id = pathParts[pathParts.length - 1];
   const isCreateMode = id === 'new';

   const [scheduledJob, setScheduledJob] = useState(null);
   const [isLoading, setIsLoading] = useState(true);
```

```
    const fetchScheduledJob = useCallback(async () => {
        if (isCreateMode) {
            setIsLoading(false);
            return;
        }
        setIsLoading(true);
        try {
            const data = await api.getScheduledJob(id);
            setScheduledJob(data);
        } catch (e) {
            Toast.error(e.message);
        } finally {
            setIsLoading(false);
        }
    }, [api, id, isCreateMode]);

    useEffect(() => {
        fetchScheduledJob();
    }, [fetchScheduledJob]);

    if (!can('link:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to manage scheduled jobs.</p></Card>;
    }

    if (!isCreateMode && isLoading) {
        return <Card title="Loading..." className="max-w-3xl mx-auto"><p>Fetching scheduled
job details...</p></Card>;
    }

    if (!isCreateMode && !scheduledJob) {
        return <Card title="Not Found" className="max-w-3xl mx-auto"><p>The requested
scheduled job does not exist or access is denied.</p></Card>;
    }

    const handleSave = () => {
        setPage('/dashboard/scheduled-jobs');
    };

    const handleCancel = () => {
        setPage('/dashboard/scheduled-jobs');
    };
```

```
    return <ScheduledJobForm scheduledJob={scheduledJob} onSave={handleSave}
onCancel={handleCancel} />;
};

const ScheduledJobListPage = () => {
    const { api, setPage, can } = useAuth();
    const [scheduledJobs, setScheduledJobs] = useState([]);
    const [isLoading, setIsLoading] = useState(true);

    const fetchScheduledJobs = useCallback(async () => {
        setIsLoading(true);
        try {
            const data = await api.getScheduledJobs();
            setScheduledJobs(data);
        } catch (e) {
            Toast.error(`Failed to load scheduled jobs: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        fetchScheduledJobs();
    }, [fetchScheduledJobs]);

    if (!can('link:create_edit')) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to view this list.</p></Card>;
    }

    const handleDelete = async (id) => {
        if (window.confirm('Are you sure you want to delete this scheduled job link?')) {
            try {
                await api.deleteScheduledJob(id);
                fetchScheduledJobs();
            } catch (e) {
                Toast.error(`Delete failed: ${e.message}`);
            }
        }
    };

    const columns = [
        { header: 'Link Name', cell: (sj) => sj.name },
        { header: 'Job', cell: (sj) => sj.jobName },
```

```
        { header: 'Schedule', cell: (sj) => sj.scheduleName },
        { header: 'Priority', cell: (sj) => <Badge variant={sj.priority === 1 ? 'critical' : sj.priority === 2
? 'info' : 'secondary'}>{PRIORITY_OPTIONS.find(p => p.value === sj.priority)?.label}</Badge>
},
        { header: 'Active', cell: (sj) => <Badge variant={sj.isactive ? 'success' :
'secondary'}>{sj.isactive ? 'Yes' : 'No'}</Badge> },
        { header: 'Created By', cell: (sj) => sj.createdby },
        {
            header: 'Actions',
            cell: (sj) => (
                <DropdownMenu trigger={<Button size="sm"
variant="secondary">Actions</Button>}>
                    {can('link:create_edit') && <DropdownMenuItem onClick={() =>
setPage(`/dashboard/scheduled-jobs/${sj.scheduledJobId}`)}>Edit</DropdownMenuItem>}
                    {can('link:create_edit') && <DropdownMenuItem onClick={() =>
handleDelete(sj.scheduledJobId)} className="text-red-600">Delete</DropdownMenuItem>}
                </DropdownMenu>
            ),
        },
    ];

    return (
        <div className="space-y-6">
            <div className="flex justify-between items-center">
                <h1 className="text-3xl font-bold text-gray-900">Scheduled Jobs</h1>
                {can('link:create_edit') && (
                    <Button onClick={() => setPage('/dashboard/scheduled-jobs/new')}>+ Link Job to
Schedule</Button>
                )}
            </div>
            <Card>
                {isLoading ? (
                    <p className="text-center text-gray-500">Loading scheduled jobs...</p>
                ) : (
                    <DataTable columns={columns} data={scheduledJobs} />
                )}
            </Card>
        </div>
    );
};

// --- MONITORING PAGES ---

const ExecutionDetailPage = () => {
```

```
    const { api } = useAuth();
    const pathParts = useAuth().page.split('/');
    const id = pathParts[pathParts.length - 1];

    const [execution, setExecution] = useState(null);
    const [isLoading, setIsLoading] = useState(true);

    const fetchExecution = useCallback(async () => {
        setIsLoading(true);
        try {
            const data = await api.getExecution(id);
            setExecution(data);
        } catch (e) {
            Toast.error(e.message);
        } finally {
            setIsLoading(false);
        }
    }, [api, id]);

    useEffect(() => {
        fetchExecution();
    }, [fetchExecution]);

    if (!execution && !isLoading) {
        return <Card title="Not Found" className="max-w-4xl mx-auto"><p>Execution record not
found or access denied.</p></Card>;
    }

    if (isLoading) {
        return <Card title="Loading..." className="max-w-4xl mx-auto"><p>Fetching execution
details...</p></Card>;
    }

    const statusVariant = execution.executionStatus === 'completed' ? 'success' :
execution.executionStatus === 'failed' ? 'critical' : 'info';

    return (
        <div className="space-y-6">
            <h1 className="text-3xl font-bold text-gray-900">Execution Details:
#{execution.executionId}</h1>

            {/* Summary Card */}
            <Card title="Summary" className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-
4">
```

```
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Status</p>
              <Badge variant={statusVariant} className="text-
base">{execution.executionStatus.toUpperCase()}</Badge>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Job Name</p>
              <p className="font-medium text-gray-900">{execution.jobName}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Scheduled Link</p>
              <p className="font-medium text-gray-900">{execution.scheduledJobName}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Exit Code</p>
              <p className="font-medium text-gray-900">{execution.exitCode}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Start Time</p>
              <p className="font-medium text-gray-
900">{formatTime(execution.startTime)}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">End Time</p>
              <p className="font-medium text-gray-900">{formatTime(execution.endTime)}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Duration</p>
              <p className="font-medium text-gray-
900">{formatDuration(execution.duration)}</p>
            </div>
            <div className="space-y-1">
              <p className="text-sm font-medium text-gray-500">Ran By</p>
              <p className="font-medium text-gray-900">{execution.createdby}</p>
            </div>
          </Card>

          {/* Log Output Card (A03 XSS Control) */}
          <Card title="Process Output" className="space-y-4">
            <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
              <div className="space-y-2">
                <h4 className="text-base font-semibold">Standard Output (stdout)</h4>
                <pre className="text-xs p-3 overflow-auto bg-gray-50 rounded-lg border font-
mono max-h-64 whitespace-pre-wrap">{execution.output || 'No stdout log.'}</pre>
```

```
                </div>
                <div className="space-y-2">
                    <h4 className="text-base font-semibold">Standard Error (stderr)</h4>
                    <pre className="text-xs p-3 overflow-auto bg-gray-50 rounded-lg border font-
mono max-h-64 whitespace-pre-wrap text-red-700">{execution.errorOutput || 'No stderr
log.'}</pre>
                </div>
            </div>
        </Card>
    </div>
  );
};

const ExecutionMonitoringPage = () => {
  const { api } = useAuth();
  const [executions, setExecutions] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [filter, setFilter] = useState({ jobId: null, status: null, query: '' });

  const deferredFilter = useDeferredValue(filter);

  const fetchExecutions = useCallback(async (currentFilter) => {
    setIsLoading(true);
    try {
      const data = await api.getExecutions(currentFilter);
      setExecutions(data);
    } catch (e) {
      Toast.error(`Failed to load executions: ${e.message}`);
    } finally {
      setIsLoading(false);
    }
  }, [api]);

  useEffect(() => {
    fetchExecutions(deferredFilter);
  }, [deferredFilter, fetchExecutions]);

  const columns = [
    { header: 'Status', cell: (e) => <Badge variant={e.executionStatus === 'completed' ?
'success' : e.executionStatus === 'failed' ? 'critical' :
'info'}>{e.executionStatus.toUpperCase()}</Badge> },
    { header: 'Job Name', cell: (e) => e.jobName },
    { header: 'Scheduled Link', cell: (e) => e.scheduledJobName },
    { header: 'Start Time', cell: (e) => formatTime(e.startTime) },
```

```
        { header: 'Duration', cell: (e) => formatDuration(e.duration) },
        { header: 'Ran By', cell: (e) => e.createdby },
        {
            header: 'Actions',
            cell: (e) => (
                <Button size="sm" onClick={() =>
api.setPage(`/dashboard/monitoring/${e.executionId}`)}>View Details</Button>
            ),
        },
    ];

    return (
        <div className="space-y-6">
            <h1 className="text-3xl font-bold text-gray-900">Execution History</h1>
            <Card>
                <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-4">
                    <Input placeholder="Search job name..." onChange={e => setFilter(f => ({ ...f,
query: e.target.value }))} />
                    <Select
                        options={[{ value: '', label: 'All Statuses' }, { value: 'completed', label: 'Completed'
}, { value: 'failed', label: 'Failed' }, { value: 'running', label: 'Running' }]}
                        onChange={e => setFilter(f => ({ ...f, status: e.target.value || null }))}
                    />
                </div>
                {isLoading ? (
                    <p className="text-center text-gray-500">Loading execution history...</p>
                ) : (
                    <DataTable columns={columns} data={executions} />
                )}
            </Card>
        </div>
    );
};

// --- ALERT PAGES ---

const AlertingPage = () => {
    const { api, can } = useAuth();
    const [alerts, setAlerts] = useState([]);
    const [isLoading, setIsLoading] = useState(true);
    const [filter, setFilter] = useState({ acknowledged: false, severity: null });

    const deferredFilter = useDeferredValue(filter);
```

```
    const fetchAlerts = useCallback(async (currentFilter) => {
        setIsLoading(true);
        try {
            const data = await api.getAlerts(currentFilter);
            setAlerts(data);
        } catch (e) {
            Toast.error(`Failed to load alerts: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        fetchAlerts(deferredFilter);
    }, [deferredFilter, fetchAlerts]);

    if (!can('alert:view_all') && alerts.length === 0 && !isLoading) {
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-
red-500">You do not have the permission to view alerts created by other users.</p></Card>;
    }

    const handleAcknowledge = async (id) => {
        try {
            await api.acknowledgeAlert(id);
            // Quick update without full re-fetch for responsiveness
            setAlerts(alerts.map(a => a.alertId === id ? { ...a, acknowledged: true, acknowledgedBy:
api.currentUser.email } : a));
        } catch (e) {
            Toast.error(`Acknowledge failed: ${e.message}`);
        }
    };

    const columns = [
        { header: 'Severity', cell: (a) => <Badge variant={a.severity === 'critical' ? 'critical' :
a.severity === 'warning' ? 'warning' : 'info'}>{a.severity.toUpperCase()}</Badge> },
        { header: 'Title', cell: (a) => a.title },
        { header: 'Scheduled Job', cell: (a) => a.scheduledJobName },
        { header: 'Date', cell: (a) => formatTime(a.createddate) },
        { header: 'Status', cell: (a) => <Badge variant={a.acknowledged ? 'secondary' :
'critical'}>{a.acknowledged ? 'Acknowledged' : 'Active'}</Badge> },
        {
            header: 'Actions',
            cell: (a) => (
```

```
            <DropdownMenu trigger={<Button size="sm"
variant="secondary">Actions</Button>}>
                {a.executionId && <DropdownMenuItem onClick={() =>
api.setPage(`/dashboard/monitoring/${a.executionId}`)}>View
Execution</DropdownMenuItem>}
                {!a.acknowledged && (can('alert:acknowledge_all') || a.createdby ===
api.currentUser.email) && (
                    <DropdownMenuItem onClick={() =>
handleAcknowledge(a.alertId)}>Acknowledge</DropdownMenuItem>
                )}
            </DropdownMenu>
        ),
    },
  ];

  return (
    <div className="space-y-6">
      <h1 className="text-3xl font-bold text-gray-900">Alert Console</h1>
      <Card>
        <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-4">
          <Select
            options={[{ value: 'false', label: 'Active Only' }, { value: 'true', label: 'Show
Acknowledged' }]}
            onChange={e => setFilter(f => ({ ...f, acknowledged: e.target.value === 'true' }))}
          />
          <Select
            options={[{ value: '', label: 'All Severities' }, ...ALERT_SEVERITY_OPTIONS]}
            onChange={e => setFilter(f => ({ ...f, severity: e.target.value || null }))}
          />
        </div>
        {isLoading ? (
          <p className="text-center text-gray-500">Loading alerts...</p>
        ) : (
          <DataTable columns={columns} data={alerts} />
        )}
      </Card>
    </div>
  );
};

// --- DASHBOARD PAGES ---

const StatCard = ({ title, value, icon: Icon, valueColor = 'text-gray-900' }) => (
  <Card className="flex items-center justify-between p-5">
```

```
      <div>
        <p className="text-sm font-medium text-gray-500">{title}</p>
        <p className={`mt-1 text-3xl font-bold ${valueColor}`}>{value}</p>
      </div>
      {Icon && <Icon className="h-8 w-8 text-indigo-400 opacity-70" />}
    </Card>
);

const ExecutionTrendCard = ({ data }) => (
    <Card title="Execution Trend (7 Days)" className="lg:col-span-2">
      <div className="flex h-48 space-x-2 items-end pt-4">
        {data.map((item, index) => {
          const total = item.status_completed + item.status_failed;
          const completedHeight = total > 0 ? (item.status_completed / total) * 100 : 0;
          const failedHeight = total > 0 ? (item.status_failed / total) * 100 : 0;

          // Max height scaling based on max total across all days for better visual comparison
          const maxTotal = data.reduce((max, current) => Math.max(max,
current.status_completed + current.status_failed), 1);
          const scaleFactor = 100 / maxTotal;

          return (
            <div key={index} className="flex flex-col items-center flex-1 h-full">
              <div className="flex-1 w-full flex flex-col justify-end items-center" >
                <div title={`Completed: ${item.status_completed}`} className="w-4 bg-green-
500 rounded-t-sm" style={{ height: `${(item.status_completed * scaleFactor * 0.9)}%` }}></div>
                <div title={`Failed: ${item.status_failed}`} className="w-4 bg-red-500
rounded-b-sm" style={{ height: `${(item.status_failed * scaleFactor * 0.9)}%` }}></div>
              </div>
              <span className="text-xs text-gray-500 mt-1">{item.day}</span>
            </div>
          );
        })}
      </div>
      <div className="flex justify-center text-xs space-x-4 mt-4">
        <div className="flex items-center space-x-1"><span className="h-2 w-2 bg-green-
500 rounded-full"></span><span>Completed</span></div>
        <div className="flex items-center space-x-1"><span className="h-2 w-2 bg-red-500
rounded-full"></span><span>Failed</span></div>
      </div>
    </Card>
);
```

```
const RecentActivityCard = ({ executions, setPage }) => (
  <Card title="Recent Activity">
    {executions.length === 0 ? (
      <p className="text-gray-500">No recent executions to display.</p>
    ) : (
      <ul className="divide-y divide-gray-200">
        {executions.map(e => (
          <li key={e.executionId} className="py-3 flex justify-between items-center
hover:bg-gray-50 cursor-pointer" onClick={() =>
setPage(`/dashboard/monitoring/${e.executionId}`)}>
            <div className="flex items-center space-x-3">
              <Badge variant={e.executionStatus === 'completed' ? 'success' :
e.executionStatus === 'failed' ? 'critical' : 'info'} className="w-2 h-2 p-0"></Badge>
              <div>
                <p className="text-sm font-medium text-gray-900">{e.jobName}</p>
                <p className="text-xs text-gray-500">{e.scheduledJobName}</p>
              </div>
            </div>
            <span className="text-xs text-gray-500">{new
Date(e.startTime).toLocaleTimeString()}</span>
          </li>
        ))}
      </ul>
    )}
    <div className="pt-4 border-t mt-4">
      <Button variant="link" size="sm" onClick={() => setPage('/dashboard/monitoring')}
className="p-0">View All Activity &rarr;</Button>
    </div>
  </Card>
);

const ActiveAlertsCard = ({ alerts, setPage }) => (
  <Card title="Active Alerts">
    {alerts.length === 0 ? (
      <p className="text-gray-500">No active alerts to display. All clear!</p>
    ) : (
      <ul className="divide-y divide-gray-200">
        {alerts.map(a => (
          <li key={a.alertId} className="py-3 flex justify-between items-center hover:bg-
gray-50 cursor-pointer" onClick={() => setPage('/dashboard/alerts')}>
            <div className="flex items-center space-x-3">
              <Badge variant={a.severity === 'critical' ? 'critical' : 'warning'} className="w-2
h-2 p-0"></Badge>
              <p className="text-sm font-medium text-gray-900">{a.title}</p>
```

```
                    </div>
                    <span className="text-xs text-red-500 font-semibold">NEW</span>
                  </li>
                ))}
              </ul>
            )}
            <div className="pt-4 border-t mt-4">
              <Button variant="link" size="sm" onClick={() => setPage('/dashboard/alerts')}
className="p-0">View All Alerts &rarr;</Button>
            </div>
          </Card>
);

const DashboardPage = () => {
    const { api, setPage } = useAuth();
    const [summary, setSummary] = useState(null);
    const [isLoading, setIsLoading] = useState(true);

    const fetchSummary = useCallback(async () => {
        setIsLoading(true);
        try {
            const data = await api.getDashboardSummary();
            setSummary(data);
        } catch (e) {
            Toast.error(`Failed to load dashboard: ${e.message}`);
        } finally {
            setIsLoading(false);
        }
    }, [api]);

    useEffect(() => {
        fetchSummary();
    }, [fetchSummary]);

    if (isLoading) {
        return (
            <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
                <Card title="Loading..." className="h-28"><div className="h-10 bg-gray-200
rounded animate-pulse"></div></Card>
                <Card title="Loading..." className="h-28"><div className="h-10 bg-gray-200
rounded animate-pulse"></div></Card>
                <Card title="Loading..." className="h-28"><div className="h-10 bg-gray-200
rounded animate-pulse"></div></Card>
```

```
            <Card title="Loading Trend..." className="lg:col-span-2 h-72"><div className="h-
full bg-gray-200 rounded animate-pulse"></div></Card>
            <Card title="Loading Lists..." className="h-72"><div className="h-full bg-gray-200
rounded animate-pulse"></div></Card>
          </div>
        );
    }

    if (!summary) {
        return <Card title="Error" className="max-w-xl mx-auto"><p className="text-red-
500">Failed to load dashboard data. Please refresh.</p></Card>;
    }

    return (
        <div className="space-y-6">
            <h1 className="text-3xl font-bold text-gray-900">System Overview</h1>
            <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
                <StatCard title="Active Schedules" value={summary.activeSchedules}
icon={Calendar} />
                <StatCard title="Failed Jobs (24h)" value={summary.failedJobs24h}
icon={AlertTriangle} valueColor={summary.failedJobs24h > 0 ? 'text-red-600' : 'text-green-600'}
/>
                <StatCard title="Active Alerts" value={summary.activeAlerts} icon={AlertTriangle}
valueColor={summary.activeAlerts > 0 ? 'text-yellow-600' : 'text-gray-900'} />
            </div>

            <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
                <ExecutionTrendCard data={summary.executionTrend} />
                <div className="space-y-4">
                    <RecentActivityCard executions={summary.recentExecutions} setPage={setPage}
/>
                    <ActiveAlertsCard alerts={summary.activeAlertList} setPage={setPage} />
                </div>
            </div>
        </div>
    );
};


const SettingsPage = () => {
    const { can } = useAuth();

    if (!can('settings:view')) {
```

```
        return <Card title="Access Denied" className="max-w-xl mx-auto"><p className="text-red-500">You do not have the permission (`settings:view`) to access system settings.</p></Card>;
    }

    return (
        <Card title="System Settings (Slice 10.0 Placeholder)" className="max-w-4xl mx-auto">
            <h2 className="text-xl font-semibold mb-4">Configuration Management</h2>
            <div className="space-y-4">
                <Input label="SMTP Host" defaultValue="smtp.harpy.com" disabled={!can('settings:manage')} />
                <Input label="Alert From Email" defaultValue="alerts@harpy.com" disabled={!can('settings:manage')} />
                <Input label="Default Timezone" defaultValue="UTC" disabled={!can('settings:manage')} />
            </div>
            {can('settings:manage') ? (
                <div className="mt-6 flex justify-end">
                    <Button>Save System Configuration</Button>
                </div>
            ) : (
                <p className="mt-4 text-sm text-yellow-600">You do not have permission (`settings:manage`) to save changes.</p>
            )}
        </Card>
    );
};


const Router = () => {
    const { page } = useAuth();
    const path = page.split('?')[0];

    if (path.startsWith('/auth')) return <AuthPage />;

    if (path.startsWith('/dashboard')) {
        const route = path.replace('/dashboard', '');

        let content = <DashboardPage />;
        if (route === '') content = <DashboardPage />;
        else if (route === '/users') content = <UserManagementPage />;
        else if (route === '/profile') content = <ProfilePage />;
        else if (route === '/credentials') content = <CredentialManagementPage />;
        else if (route.startsWith('/jobs')) content = <JobPage />;
```

```
        else if (route.startsWith('/schedules')) content = <SchedulePage />;
        else if (route.startsWith('/scheduled-jobs')) content = <ScheduledJobPage />;
        else if (route.startsWith('/monitoring')) content = path.split('/').length > 3 ?
<ExecutionDetailPage /> : <ExecutionMonitoringPage />;
        else if (route === '/alerts') content = <AlertingPage />;
        else if (route === '/settings') content = <SettingsPage />;
        else content = <div className="text-center p-10">404 - Page Not Found</div>;

        return <AppLayout>{content}</AppLayout>;
    }

    return <div className="text-center p-10">404 - Page Not Found</div>;
};

const App = () => (
    <AuthProvider>
        <Router />
    </AuthProvider>
);

export default App;
```